

Evaluating recalibration strategies for real-time crash prediction: adaptive drift detection vs. cumulative retraining

Franco Basso^{a,b,*}, Vicente Fernández-Figueroa^a, Tomás Maldonado^c, Diego A. Martínez^{a,d}, Raúl Pezoa^c, Mauricio Varas^e, Felipe Villa^a

^a*School of Industrial Engineering, Pontificia Universidad Católica de Valparaíso, Valparaíso, Chile*

^b*Instituto Sistemas Complejos de Ingeniería, Chile*

^c*Department of Industrial Engineering, Universidad Diego Portales, Santiago, Chile*

^d*Department of Emergency Medicine, Johns Hopkins University School of Medicine, Baltimore, Maryland, USA*

^e*Centro de Investigación en Sustentabilidad y Gestión Estratégica de Recursos, Facultad de Ingeniería, Universidad del Desarrollo, Santiago, Chile*

Abstract

Accurate crash prediction is vital for road safety, but sustaining model performance over time remains challenging due to evolving traffic conditions and shifting data distributions. Despite advances in developing predictive models, how and when to recalibrate these models has received limited attention in crash prediction research. This study addresses this gap by evaluating [six](#) recalibration strategies: static, period aligned, cumulative, and [three](#) adaptive drift detection [approaches](#): using ADWIN (Adaptive Windowing), [Adaptive Random Forest \(ARF\)](#), and [KSWIN \(Kolmogorov–Smirnov Windowing\)](#), applied to crash prediction models (XGBoost, AdaBoost, Random Forest, and NNet) using Automatic Vehicle Identification data from Santiago, Chile (2018–2024). [Class imbalance is addressed through SMOTE-based oversampling, and hyperparameters are tuned via Bayesian optimization with Optuna.](#) Unlike time-based approaches, ADWIN automatically detects concept drift by monitoring prediction error distributions; [ARF adapts continuously online without external batch retraining;](#) and [KSWIN detects covariate drift by monitoring feature distributions,](#) enabling updates only when statistically significant changes occur. Results show ensemble models, especially XGBoost and AdaBoost, offer the best tradeoff between accuracy, calibration, and computational cost. Cumulative retraining emerges as most effective, likely because crash prediction’s rare event nature benefits from maximizing historical data. While [the ADWIN family of approaches offers](#) a theoretically appealing adaptive approach, [their](#) performance suggests drift detection methods require further refinement for rare event contexts. These findings provide the first systematic comparison of recalibration strategies for crash prediction, demonstrating that rare event characteristics favor data accumulation over adaptive drift detection.

Keywords: Crash prediction, recalibration, drift detection, machine learning, predictive performance.

1. Introduction

The sustained increase in vehicular traffic on urban highways has generated significant negative externalities, with traffic accidents among the most critical (?). These accidents not only pose a direct threat

*Corresponding author

Email addresses: francobasso@gmail.com (Franco Basso), vicente.fernandez.f@mail.pucv.cl (Vicente Fernández-Figueroa), tomas.maldonado1@mail.udp.cl (Tomás Maldonado), diego.martinez.c@pucv.cl (Diego A. Martínez), raul.pezoa@udp.cl (Raúl Pezoa), mavaras@udd.cl (Mauricio Varas), felipe.villa.r@mail.pucv.cl (Felipe Villa)

to human life, but also generate substantial economic effects, including traffic congestion, productivity losses, and high social costs (???). In Chile, for example, more than 75,653 road accidents were recorded in 2024, resulting in 1,439 fatalities. Although this represents a decrease of approximately 12% compared to 2023, the economic impact remains substantial: the social cost of traffic accidents was estimated at 2.21% of the national Gross Domestic Product in 2023, equivalent to over US\$6.63 billion (?).

Intelligent Transportation Systems (ITS) have become an effective means to reduce traffic accidents and improve the operational efficiency of urban highways (?). By integrating advanced technologies such as real-time sensors, vehicular networks, predictive analytics, and machine learning algorithms (?), ITS support data-driven decision-making in road traffic management (?). A prominent example is real-time crash prediction (RTCP), which uses statistical and machine learning models together with disaggregated traffic speed and flow data to anticipate accident occurrence (?). These systems enable the early identification of crash-prone conditions and the deployment of preventive measures, including dynamic alerts (??) and traffic flow adjustments (?), thereby helping to reduce accident frequency and severity while enhancing safety in complex, dynamic urban settings (???). A wide range of machine learning and deep learning techniques have been applied to crash prediction on highways, drawing on diverse data sources such as loop detectors (?), video cameras (?), speed radars, and Automatic Vehicle Identification (AVI) systems (?).

Despite significant advancements, the performance of artificial intelligence (AI) techniques for crash prediction can decline over time as changes in driving patterns, seasonal variations, regulatory updates, or extraordinary events alter the underlying data distribution and weaken predictive accuracy (?). In such dynamic contexts, recalibration is essential to sustain model effectiveness (??) and ensure continued value for real-time decision-making. This challenge is particularly relevant for Autopista Central, a major urban highway in Santiago, Chile, where a pilot RTCP system developed by ? is in operation. A key factor behind performance degradation is *drift*, the gradual or abrupt change in the statistical properties of input or output variables over time, which can disrupt the relationship between predictors and the target variable (?). While drift detection methods have shown effectiveness in other domains, they remain unexplored in this context. More broadly, there is no prior research comparing alternative recalibration strategies within operational online crash prediction systems, despite the need to sustain accuracy under evolving traffic patterns while controlling computational demands.

In this paper, we address the challenge of recalibrating accident prediction models in non-stationary, dynamic traffic environments, using multi-year real-world data (2018–2024) from Autopista Central in Santiago, Chile. The highway’s Automatic AVI gate system generates over 600 million records annually—more than 1.6 million daily observations—placing substantial demands on data processing and model updating. In such contexts, recalibration must balance sustaining predictive accuracy under evolving traffic patterns with minimizing the computational time, resource use, and operational complexity of frequent updates. To address this, we contribute to the literature by applying a set of recalibration strategies—static, period-aligned, cumulative, and [three adaptive strategies \(ADWIN, Adaptive Random Forest, and KSWIN\)](#)—to RTCP models built with XGBoost, AdaBoost, Random Forest, and Neural Networks, using AVI data from Autopista Central. [Class imbalance is handled via SMOTE-based oversampling, and hyperparameters are optimized using Bayesian search with Optuna.](#) We then conduct extensive computational experiments to compare the efficiency and predictive performance of these strategies, highlighting their respective advantages and limitations.

The structure of this article is as follows: Section 2 we position our investigation in the state of art with a extensive literature review. Section 3 describes the characteristics of the dataset used, including data sources, initial preprocessing, and the variables considered. Section 4 outlines the proposed methodology, detailing the recalibration approaches evaluated, the integration of [three adaptive drift detection methods \(ADWIN, ARF, and KSWIN\)](#), and the training procedures. Section 5 presents the computational experiments, the results obtained, and a comparative analysis of the methodologies. Finally, Section 6 discusses

the study’s conclusions and proposes future research directions.

2. Literature Review

This section reviews the literature that supports this study. Specifically, Subsection 2.1 reviews recent advances in crash prediction models, focusing on statistical and machine learning approaches for highways and urban intersections. Subsection 2.2 discusses the challenges of maintaining model performance in dynamic settings, with particular attention to data and concept drift. Subsection 2.3 presents applications of drift detection in transportation, highlighting their effectiveness and current limitations. Finally, Subsection 2.4 identifies the research gap this study aims to address.

2.1. Crash Prediction Models

The first studies in RTCP focus on highways using statistical and machine learning methods. ? proposed a hybrid model that combines multinomial logit regression with Bayesian Belief Networks to identify key predictors and forecast collisions on the Tokyo Metropolitan Expressway. ? developed a Support Vector Machine model to analyze factors associated with accidents on the Autopista Central in Santiago, Chile. Expanding on this work, ? introduced a deep learning model that uses vehicle-level data and image-based architectures such as CNN and GAN to capture real-time driving patterns on the same expressway. More recently, ? applied a log-logistic model to Indian Ministry of Transport data, highlighting the predictive value of traffic density, prior accidents, weather, and time of day.

Crash prediction at urban intersections and segments has recently received growing attention, driven by the increasing availability of detailed traffic data (?). These locations often experience a high number of collisions due to their structural complexity and dynamic traffic patterns. ? proposed a model for intersections in Victoria, Australia, combining crash reports, roadworks, and environmental data within an ensemble decision tree framework. In Canada, ? developed a real-time prediction model for signalized intersections using the Generalized Extreme Value (GEV) distribution, a method commonly used in intersection crash analysis due to its ability to model rare and extreme events. More recent studies have incorporated data from connected vehicles. For example, ? applied a Geographical Random Forest to large-scale datasets from the Safety Pilot Model Deployment in Ann Arbor, USA. ? introduced RoadInTCP, a deep learning model for sequential, multimodal crash prediction at intersections, which was validated using traffic data from New York City.

ITS increasingly incorporate predictive modeling to enhance road safety and support operational decision-making (??). Within this framework, predictive models provide critical capabilities for identifying high-risk conditions, guiding infrastructure investment, and improving emergency preparedness. For example, ? developed an intelligent accident crisis management system that combines Modified Convolutional Neural Networks (MCNN) with Particle Swarm Optimization (PSO) to improve both crash detection and response efficiency through vehicle trajectory analysis. Similarly, ? and ? examined the integration of machine learning into traffic emergency management frameworks, emphasizing its potential to optimize real-time responses and allocate resources more effectively in transportation systems.

However, as predictive models become increasingly embedded into real-time traffic management and decision-support systems, a key operational challenge remains unresolved: how to ensure their sustained performance in the face of evolving traffic dynamics. Most existing implementations rely on static or periodically updated models, often lacking mechanisms to detect and respond to changes in data distributions. This shortcoming limits their long-term effectiveness, particularly in non-stationary environments such as urban highways or intersections, where traffic patterns, behaviors, and risks can shift rapidly. As noted in ?, even the calibration factors used in standardized safety models can become unreliable over time if not regularly updated with sufficient data and methodological rigor.

2.2. Recalibration

Machine learning models deployed in real-world applications, such as traffic prediction or real-time flow monitoring, face the ongoing challenge of maintaining performance over time (?). A common approach to address this issue is recalibration, which involves periodically updating or retraining the model using new data samples (?). Recalibration typically depends on prediction quality and observed data shifts, enabling the model to adapt as conditions change (?), or is triggered after a sufficient volume of new data becomes available (?).

One of the main drivers behind the need for recalibration is the emergence of changes in the data known as drift, that is, alterations in the statistical distribution of the data over time (?). These changes are inevitable in dynamic environments such as urban traffic, sensor monitoring, or financial systems, and may manifest in different forms. The first and most common is data drift, also referred to as covariate shift (?), which occurs when the marginal distribution of the input variables changes without necessarily altering their relationship with the target variable.

On the other hand, within the domain of drift, concept drift emerges as a critical phenomenon in machine learning systems deployed in dynamic environments, where the distribution of input data evolves over time, negatively impacting predictive performance (?). This issue is particularly relevant in applications that process real-time data streams, such as traffic prediction, fraud detection, or sensor analytics in IoT environments (?). To address this challenge, several methodologies have been developed to detect and adapt models to these changes, preserving their generalization capability.

Seminal works in this field laid the foundation for modern approaches. ? introduced the FLORA algorithm, a rule-based incremental learning method for binary classification with nominal data, designed to handle concept drift. It models each concept through three sets of rule antecedents: ADES (positive examples), NDES (negative examples), and PDES (potential descriptions matching both classes). ADES classifies new data, NDES prevents overgeneralization, and PDES stores descriptions that may become relevant later, with counters tracking coverage in a sliding window. ? introduce a pioneering technique based on monitoring the model’s online error rate. By detecting statistically significant deviations in this rate, the system can infer that a shift in the underlying distribution has occurred and act accordingly. This method proved especially effective in online learning scenarios, where immediate feedback enables timely adaptation.

Subsequently, ? developed one of the most influential algorithms to date: ADWIN (Adaptive Windowing). This detector dynamically adjusts the size of a sliding window over the data stream to identify concept changes without requiring a fixed window size or a predefined threshold. Its computational efficiency, ease of implementation, and effectiveness in non-stationary environments have positioned it as a central reference in the drift detection literature. The importance of data drift detectors has been further solidified through multiple contemporary validations. Around the same time, ? proposed the OLINDDA algorithm (OnLine Novelty and Drift Detection Algorithm), which models the normal concept from a single class (one-class classification) and monitors unrecognized data to detect the formation of new clusters.

In recent years, studies such as ?, ?, ?, ?, and ? have shown that these methods not only preserve predictive performance but also minimize computational overhead and reduce the need for frequent re-training. This progress makes them particularly valuable in applications where computational resources are limited or latency is critical.

2.3. Implementation of Drift Detection in Transport

While concept and data drift detection techniques have seen rapid development in domains such as finance and e-commerce, their deployment in transportation applications is comparatively recent. A growing body of work now demonstrates how adaptive drift-handling mechanisms can preserve model performance and resilience within highly dynamic mobility environments.

Early real-time traffic studies illustrated the value of built-in adaptation. For instance, ? employed FIMT-DD, an incremental model tree with native drift detection, to forecast large-scale highway flow in the UK, maintaining accuracy under temporal shifts. Similarly, ? developed the BRIGHT ensemble for short-term taxi demand forecasting across Portugal, China, and Sweden, integrating drift-aware base learners and bias correction to respond to rapidly changing urban conditions. Subsequent work extended adversarial and transport-theoretic approaches. ? combined adversarial validation with gradient-boosted decision trees to detect and correct distributional shifts in Uber user targeting across the Asia-Pacific region. ? introduced OTDD, using Optimal Transport distances to identify drift in mobility mode classification using smartphone and vehicular sensor streams.

In continuous stream and edge/fog settings, adaptive windowing and unsupervised signal-based detectors have proven effective. ? used an auto-tuned ensemble (PWPAE) on a MNIST-derived vehicular traffic simulation to showcase windowed drift adaptation, while ? applied similar window-based strategies for network traffic streams in edge/fog infrastructures. ? further demonstrated spectral-entropy drift detection on real-world transport network communications, highlighting the practicality of unsupervised methods in monitoring high-volume data.

Drift detection has also been explored in autonomous vehicle control and encrypted traffic analytics. ? proposed the OSWDD sliding-window detector alongside regression models to adapt to non-stationary conditions in simulated AV systems. ? investigated robustness of CNN/LSTM classifiers under encrypted mobile-traffic drift by combining drift-impact analysis with robust re-tuning strategies. In safety-critical environments, ? examined data drift effects on crash-risk and traffic prediction using multi-sensor fusion (camera, LiDAR, radar), highlighting the operational risks of unaddressed drift in vehicle safety systems.

Building on these advances, the present work implements ADWIN, a statistically principled adaptive windowing detector, to enable automated recalibration of batch-trained neural networks for RTCP from AVI sensor streams.

2.4. Research gap

Based on the analysis of recent literature, a sustained evolution can be observed in crash prediction models applied to highways, which have increasingly incorporated advanced technologies such as deep learning architectures, spatial networks, and connected vehicle data. However, a critically underexplored dimension concerns the recalibration of predictive models in non-stationary environments. Specifically, there is a notable lack of studies that clearly define when recalibration should occur, whether periodically or through post-hoc strategies triggered by model performance or composite metrics. Most current studies assume static models or defer recalibration to manual or heuristic processes, lacking systematic criteria or empirical validation regarding their timing, frequency, or effectiveness. Instead, they typically evaluate supervised model performance over evolving datasets without addressing distributional changes.

Despite recent advances, most applications to date have focused on traffic flow, demand forecasting, user profiling, or mobility analytics, often relying on synthetic or narrowly scoped datasets. As illustrated in Table 1, concept and data drift have been explored in various transportation contexts, from urban mobility to autonomous vehicles and transportation networks. However, the explicit use of drift detection as a principled trigger for model recalibration remains virtually unexplored in real-world crash prediction scenarios. Notably, none of the studies reviewed address real-time recalibration for crash risk estimation using highway-level operational data.

Therefore, the present study directly addresses this gap by systematically comparing several recalibration strategies, static, periodic, and cumulative retraining, against an adaptive recalibration framework guided by concept drift detection via the ADWIN algorithm. These strategies are evaluated in terms of adaptability to distributional changes, predictive performance, and computational efficiency.

Table 1: Summary of related works on concept drift (CD) and data drift (DD) in the transportation field.

Paper	Objectives	Data source	Drift-Handling Methodology
?	Aims to maintain traffic flow prediction accuracy on UK highways under non-stationary conditions using an incremental model tree (FIMT-DD) with built-in drift detection that continuously updates the model online.	Highways Agency, United Kingdom	Built-in drift detection in FIMT-DD
?	Tackles highly dynamic urban taxi demand forecasting with BRIGHT, an adaptive ensemble that retrains its base learners whenever internal drift-aware mechanisms detect changes, using GPS data.	Porto Taxi Dataset: Portugal; T-Drive (Microsoft + Tsinghua): China; Stockholm Taxi: Sweden	Adaptive ensemble with drift-aware base models
?	Addresses distribution shifts in Uber user targeting by combining adversarial validation with GBDT, where adversarial techniques detect discrepancies between training and production distributions to trigger recalibration.	Internal records from Uber Technologies Inc., Asia-Pacific region	Adversarial validation to detect drift
?	Manages drift in transport mode classification from smartphone IMU signals using OTDD , which compares input distributions over time and retrains classifiers when a shift is detected.	University of Bologna (TEMPESTA group), Italy; Ford Motor Co. & UMTRI, United States; synthetic datasets generated by the authors	OTDD
?	Handles evolving traffic-like patterns using PWPAE over synthetic MNIST (Modified National Institute of Standards and Technology) proxies, applying a sliding window to reweight classifiers in response to detected drift.	Synthetic data derived from MNIST	PWPAE ensemble adaptation with windowed drift detection
?	Deals with evolving traffic streams in edge/fog environments through incremental learning models using window-based adaptation, enabling continuous model adjustment on real-time urban traffic data.	Traffic streams captured through edge/fog gateways installed by the academic-industrial consortium in a European city (Italy/EU)	Window-based drift adaptation
?	Focuses on detecting drift in vehicular network communications by applying an unsupervised spectral entropy-based detector, which flags shifts based on frequency domain variations without labeled data.	ITS communication traces from the intelligent vehicular network in Tehran, Iran (Iran University of Science & Technology)	Spectral-entropy-based drift detection
?	Targeted non-stationary traffic in autonomous vehicle systems using a regression model combined with OSWDD , which updates model parameters dynamically upon drift detection.	Simulated scenarios on an AVS platform developed in South Korea (KAIST & collaborators)	OSWDD
?	Aimed to classify encrypted mobile traffic under data drift using a deep CNN/LSTM architecture, reinforced with hyperparameter retuning based on drift impact analysis.	Encrypted LTE traffic captures from a national mobile operator in Iran	Drift-impact analysis with robust model re-tuning
?	Addressed crash risk prediction in safety-critical vehicular environments using Deep LSTM over fused camera, LiDAR, and radar data, evaluating the need for recalibration in the presence of sensor drift.	Instrumented vehicles from UMTRI, United States (University of Michigan Transportation Research Institute)	Empirical data drift analysis
This Paper	When and how to recalibrate RTCP Models	AVI network Autopista Central, Santiago - Chile	ADWIN adaptive windowing drift detector

3. Data Description

To train the predictive models for RTCP, we use multiple data sources provided by the Autopista Central in Chile. The study period spans from January 1, 2018 to September 30, 2024, covering the full 24-hour cycle between 00:00 and 23:59 each day. First, in Subsection 3.1, we describe the traffic flow data

captured by the AVI gates. Then, in Subsection 3.2, we provide a description of the manually recorded accident data.

3.1. AVI Data

The data used in this study was provided by Autopista Central in Santiago, Chile, a major highway running north–south across the Metropolitan Region. It spans 62.3 kilometers and connects the Américo Vespucio ring road with the Maipo River. The infrastructure consists of two main segments: the General Velásquez branch and the Ruta 5 highway. This study focuses exclusively on the Ruta 5 segment (see Figure 1).

The analysis is based on an extensive dataset of vehicular traffic collected through the network of AVI’s installed along the highway. These sensors operate continuously and automatically, recording individual vehicle passages and generating detailed, high-frequency traffic data. In total, 3,618,662,236 records were obtained, each corresponding to a single vehicle crossing an AVI gate. Each row in the dataset includes attributes such as timestamp, speed, license plate, gate ID, lane number, and vehicle type. Vehicles are categorized into three classes: light, heavy, and motorcycle (see Table 2).

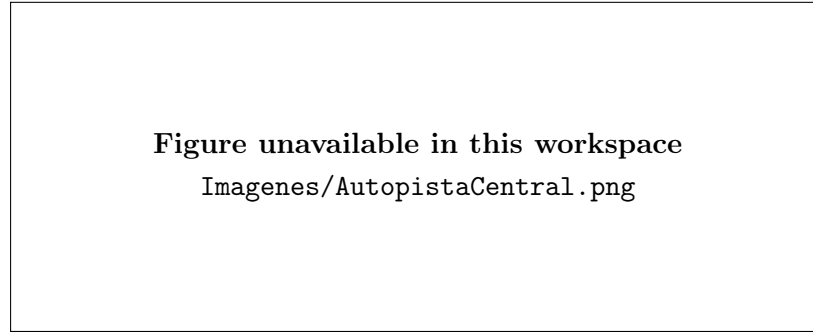


Figure 1: Autopista Central, Santiago, Chile (Source: autopistacentral.cl/sobre-autopista/)

Table 2: Sample of vehicle records captured by AVI

Timestamp	Speed (km/h)	Category	License Plate	Gateway	Lane
01/01/2018 00:41:11	111	Light	JD****	AC-01	3
01/01/2018 00:41:12	102	Light	JY****	AC-08	2
01/01/2018 00:41:14	87	Light	CC****	AC-09	2
01/01/2018 00:41:15	109	Heavy	VX****	AC-03	1
01/01/2018 00:41:13	89	Heavy	GD****	AC-07	1
01/01/2018 00:41:14	105	Motorcycle	FC****	AC-09	2

For operational management purposes, the Ruta 5 highway is divided into segments, each of which includes two roadways: Westbound and Eastbound, corresponding to the north–south and south–north directions, respectively. For the purposes of this study, we focus specifically on the segment with the highest number of crashes per kilometer, located between Carlos Valdovinos and the Mapocho River in the north–south direction (Westbound roadway). This section covers approximately 4.7 kilometers and is delineated by gates AC-09 and AC-08, respectively. To ensure full coverage of this segment, given the absence of direct monitoring between Carlos Valdovinos and AC-08, and between the Mapocho River and AC-09, the analysis is extended to include adjacent gates AC-07 and AC-10 (see Figure 2). The physical distances between these consecutive gates are 2.89 km (between AC-10 and AC-09), 2.31 km (between AC-09 and AC-08), and 3.14 km (between AC-08 and AC-07), totaling an extended segment of approximately 8.34 km.

For this segment, we have 154,559,603 records from gate AC-09 and 155,684,078 records from gate AC-08 within the considered time range. Table 3 presents an annual summary of the processed traffic volume, its composition by vehicle type, and lane distribution. In this table, *Vehicular Flow* refers to the total number of vehicles detected passing through gates AC-08 and AC-09, while *License Plates* indicates the number of distinct and valid license plates recorded by the AVI system. A high proportion of light vehicles is observed throughout the entire period, with an average of 91.59%, followed by heavy vehicles (5.68%) and motorcycles (2.73%). Regarding lane usage, Lane 2 (center lane) exhibits the highest occupancy on average (37.86%), while Lane 1, typically used by heavy or slower vehicles, accounts for 30.71%. Lane 3, commonly used by lighter and overtaking vehicles, averages 31.43% usage, with a maximum of 34.00% in 2019. In terms of speed, average values exceed 70 km/h across all categories, with Lane 1 consistently showing the lowest speeds for heavy vehicles (ranging from 65.00 to 71.68 km/h). A consistent temporal pattern is observed across years, with a peak in average speed during 2020, likely due to reduced traffic volumes under COVID-19 mobility restrictions, followed by a decline in 2021 and a gradual recovery thereafter (??). Additionally, the column *Number of Days with Data* indicates the number of days with available records for each year, which does not always match the expected 365 days due to technical issues in the database or operational failures in the AVI system of the gates, which may have prevented continuous data logging.

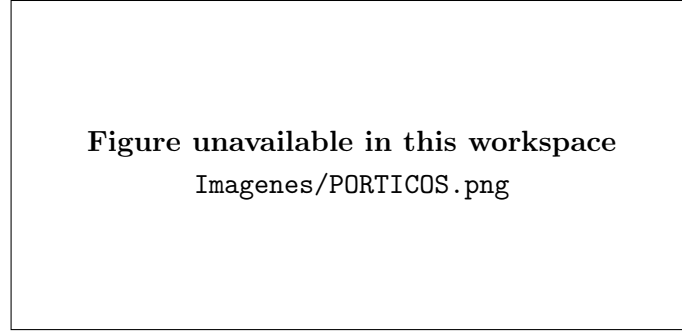


Figure 2: Autopista Central gates. Source: Adapted from ?.

Table 3: Annual summary of traffic volume, composition, and average speed by category for Gates AC-09 and AC-08 (2018–2024)

Year	Days with Data	Vehicular Flow	% Traffic Volume			License Plates	% Lane Distribution			Average Speed (Km/h)		
			Light	Heavy	Motorcycles		Lane 1	Lane 2	Lane 3	Light	Heavy	Motorcycles
2018	365	48,248,103	91.85%	5.91%	2.24%	1,975,146	29.27%	37.05%	33.68%	76.49	68.70	78.99
2019	365	49,249,559	91.93%	5.61%	2.46%	2,020,659	29.40%	36.60%	34.00%	75.02	67.12	77.57
2020	366	39,442,755	90.57%	6.37%	3.06%	1,815,218	31.59%	38.62%	29.79%	79.58	71.68	81.70
2021	363	47,353,228	91.24%	5.80%	2.96%	2,077,121	31.60%	38.49%	29.91%	72.49	65.00	75.46
2022	365	50,519,643	91.73%	5.30%	2.97%	2,224,542	30.75%	38.53%	30.72%	73.36	65.28	76.63
2023	289	39,075,208	91.80%	5.40%	2.80%	2,062,299	31.06%	37.84%	31.09%	74.11	65.83	78.00
2024	274	36,332,477	92.03%	5.38%	2.59%	1,952,830	31.29%	37.88%	30.84%	75.38	67.29	79.70
Average	341	44,317,281.86	91.59%	5.68%	2.73%	2,018,259.29	30.71%	37.86%	31.43%	75.20	67.27	78.29
Std. Dev.	40.89	5,808,935.99	0.52%	0.38%	0.30%	126,047.63	0.98%	0.78%	1.71%	2.34	2.33	2.06

3.2. Accident Data

Autopista Central has provided a total of 9,984 accident records corresponding to the period between January 2018 and September 2024. This database constitutes the official source of incident events on the roadway and is manually recorded by the concessionaire’s operational staff. Each accident is documented with three key attributes: the date and time of the incident, the segment of the highway where it occurred, and the corresponding roadway, specifying whether the event took place in the Eastbound or Westbound direction. This information enables both spatial and temporal georeferencing of each event, allowing integration with traffic flow data. For the purposes of this study, we focus on the previously described segment of the highway, which accounts for a total of 1,152 recorded accidents.

The hourly distribution of accidents in the previously mentioned segment reveals a significant concentration between 12:00 and 14:59, the time window with the highest percentage of accidents. Specifically, the intervals from 12:00–12:59 and 13:00–13:59 account for 8.7% and 8.1% of the total number of accidents, respectively, as shown in Figure 3.

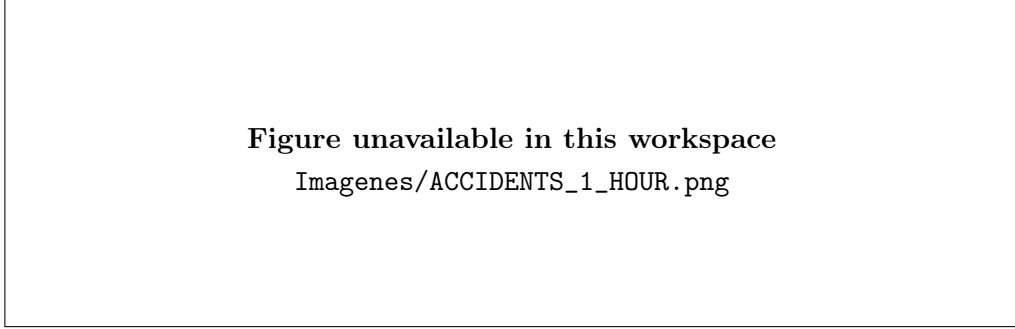


Figure 3: Accidents per hour

4. Methodology

This section presents the methodology implemented to develop and evaluate RTCP models, including a description of concept drift and the recalibration strategies used to maintain model performance over time. The structure is organized into seven main components. First, in Subsection 4.1, we present the feature engineering process, in which explanatory variables are constructed from raw data, including transformations and aggregations. Then, in Subsection 4.2, we describe the data preparation process, including various analyses of missing values in both the target and predictor variables. In Subsection 4.3, we detail the variable selection procedure, which includes correlation analysis and the application of the Random Forest algorithm for feature selection. In Subsection 4.4, we introduce the concept of drift and its relevance to non-stationary traffic environments. In Subsection 4.5, we describe the four classification models employed in this study: XGBoost, AdaBoost, Random Forest, and Neural Networks. In Subsection 4.6, we present the performance indicators used to evaluate model effectiveness, including AUC, sensitivity, specificity, and error rate. Finally, in Subsection 4.7, we outline the experimental design, detailing four distinct recalibration strategies: static (no recalibration), period-aligned retraining, cumulative recalibration, and adaptive recalibration via drift detection using the ADWIN algorithm.

4.1. Feature engineering

The *feature engineering* process constitutes a fundamental stage for transforming raw data into structured and representative variables that feed the predictive models (?). In this study, we start from the individual vehicle records captured by the electronic toll gates of the Autopista Central, as described in Subsection 3.1. The final dataset includes information from four toll gates, defined as the set $\mathcal{P} = \{\text{AC-10, AC-09, AC-08, AC-07}\}$. Each gate has three lanes enabled for vehicle flow, represented by the set $\mathcal{L} = \{1, 2, 3\}$, and three vehicle categories are considered, grouped in the set $\mathcal{C} = \{\text{Light, Heavy, Motorcycle}\}$. Throughout the variable development, the letters $x \in \mathcal{C}$ will be used to refer to the vehicle category, $Y, Z \in \mathcal{P}$ to denote the different gates, and $\ell \in \mathcal{L}$ to denote the origin lane.

For data aggregation, a fixed time interval of 5 minutes is defined ($\Delta t = 5 \text{ min}$), over which the explanatory variables of interest are computed. For each time interval t and each gate Y , the available observations are aggregated using statistical measures. The average speed of category x at gate Y during interval t is defined in Equation 1, where v_i is the recorded speed of vehicle i belonging to category x at gate Y , and $n_{x,Y,t}$ is the total number of vehicles of that category within the interval. These aggregated

statistics allow us to characterize traffic patterns and identify critical time windows, such as rush hours or low-demand periods, where variations in speed may influence accident risk.

$$\text{Vel}_x^Y(t) = \frac{1}{n_{x,Y,t}} \sum_{i=1}^{n_{x,Y,t}} v_i \quad (1)$$

Similarly, the following metrics are computed for each category x at gate Y and time interval t . The *flow* is defined in Equation 2, which corresponds to the vehicle count in interval t .

$$\text{Flow}_x^Y(t) = n_{x,Y,t} \quad (2)$$

The *vehicle density* is given in the Equation 3.

$$\text{Den}_x^Y(t) = \frac{n_{x,Y,t}}{\text{Vel}_x^Y(t)} \quad (3)$$

The temporal change in average speed is expressed in Equation 4, and similar expressions can be written for flow and density.

$$\Delta \text{Vel}_x^Y(t) = \text{Vel}_x^Y(t) - \text{Vel}_x^Y(t-1) \quad (4)$$

Then, the *speed standard deviation* is calculated in Equation 5.

$$\text{Sd}_x^Y(t) = \sqrt{\frac{1}{n_{x,Y,t}-1} \sum_{i=1}^{n_{x,Y,t}} (v_i - \text{Vel}_x^Y(t))^2} \quad (5)$$

In cases where it is possible to identify the same vehicle crossing two consecutive gates Y and Z within a reasonable time threshold (defined based on the physical distance and a minimum reference speed), the *spatial average speed* is computed as in Equation 6, where $d_{Y,Z}$ is the distance between gates Y and Z , and Δt_i is the travel time for vehicle i between both points.

$$\text{Vel}_x^{Y,Z}(t) = \frac{d_{Y,Z}}{\Delta t_x^{Y,Z}(t)} \quad (6)$$

Based on the previous definition, *flow*, *standard deviation* (Sd), and *density* (Den) are computed in the same manner as in Equation 2, Equation 5, and Equation 3, respectively, but using spatial speeds between gates Y and Z .

The **lane change ratio** was also calculated, as defined in Equation 7, where $\mathbb{I}[\bullet]$ is the indicator function, which takes the value 1 if the condition inside the brackets is met, and 0 otherwise. On the other hand, $n_x^{Y,Z}(t)$ corresponds to the total number of matched vehicles for that category in the interval. This formulation enables the aggregate quantification of lane-changing maneuvers, which may be correlated with more aggressive driving behaviors or congestion situations.

$$\text{CL}_x^{Y,Z}(t) = \frac{1}{n_x^{Y,Z}(t)} \sum_{i=1}^{n_x^{Y,Z}(t)} \mathbb{I}[\text{Lane}_i^Y \neq \text{Lane}_i^Z] \quad (7)$$

In addition to the category-stratified segment variables, *segment-global* aggregates are computed by pooling all matched vehicles across categories for each segment (Y, Z) at interval t :

$$M_{t,Y,Z} = \bigcup_{x \in \mathcal{C}} M_{x,t,Y,Z}.$$

From $M_{t,Y,Z}$, the global speed $\text{Vel}^{Y,Z}(t)$, standard deviation $\text{Sd}^{Y,Z}(t)$, density $\text{Den}^{Y,Z}(t)$, flow $\text{Flow}^{Y,Z}(t)$, and their first temporal differences $\Delta\text{Vel}^{Y,Z}(t)$, $\Delta\text{Flow}^{Y,Z}(t)$, $\Delta\text{Den}^{Y,Z}(t)$ are constructed following the same formulas in Equation 1–Equation 3, aggregated over all categories.

Origin-lane variables further disaggregate each segment by the upstream lane ℓ of the matched vehicle:

$$M_{t,Y,Z,\ell} = \{m \in M_{t,Y,Z} : \text{Lane}_m^Y = \ell\}, \quad \ell \in \mathcal{L}.$$

For each combination (Y, Z, ℓ) , the same four base statistics (Flow, Vel, Sd, Den) and three temporal differences (ΔFlow , ΔVel , ΔDen) are computed. This disaggregation captures lane-specific heterogeneity that has been identified as relevant in prior feature-importance analyses (see Figure 6).

Finally, variables representing the fraction of motorcycles and the fraction of heavy vehicles per gate Y were included, calculated as in Equation 8 and Equation 9.

$$\text{Ft}_{\text{Motorcycle}}^Y(t) = \frac{\text{Flow}_{\text{Motorcycle}}^Y(t)}{\text{Flow}_{\text{Light}}^Y(t) + \text{Flow}_{\text{Motorcycle}}^Y(t) + \text{Flow}_{\text{Heavy}}^Y(t)} \quad (8)$$

$$\text{Ft}_{\text{Heavy}}^Y(t) = \frac{\text{Flow}_{\text{Heavy}}^Y(t)}{\text{Flow}_{\text{Light}}^Y(t) + \text{Flow}_{\text{Motorcycle}}^Y(t) + \text{Flow}_{\text{Heavy}}^Y(t)} \quad (9)$$

Additionally, a binary label associated with the occurrence of accidents was integrated into the set of variables, with the aim of constructing the target variable for the prediction models. For this purpose, the accident database provided by Autopista Central, previously described in Subsection 3.2, was used. This information was aligned with the time intervals defined in our feature dataset ($\Delta t = 5$ minutes), by applying a floor rounding to each timestamp in order to correctly identify the corresponding interval. Then, both data sources were merged using the combination of date and interval start time, generating the binary variable $y(t)$, defined in Equation 10:

$$\text{Acc}(t) = \begin{cases} 1 & \text{if at least one accident occurred during interval } t + 1, \\ 0 & \text{otherwise.} \end{cases} \quad (10)$$

Based on these dimensions and the multiple variables generated through statistical aggregations, temporal and spatial differences, fractions, and lane changes, **a total of 260 predictive variables are constructed** for each observation time point, distributed as follows: (i) 96 gantry-by-category variables (4 metrics + 4 deltas $\times$ 4 gates $\times$ 3 categories); (ii) 8 compositional gantry shares (motorcycle and heavy fractions $\times$ 4 gates); (iii) 72 segment-by-category variables (5 base metrics + 3 deltas $\times$ 3 segments $\times$ 3 categories); (iv) 21 segment-global variables (4 base metrics + 3 deltas $\times$ 3 segments); and (v) 63 origin-lane variables (4 base metrics + 3 deltas $\times$ 3 segments $\times$ 3 lanes). It is worth noting that many of these variables capture heterogeneity across gates, vehicle categories, and lanes, which enhances their explanatory power and allows relevant nuances of traffic flow to be captured. The complete dataset spans a total of **686,512** five-minute periods, corresponding to the study interval between January 2018 and September 2024.

All these variables allow us to capture not only the average state of the traffic flow, but also its dynamics and heterogeneity across gates and vehicle categories. Table 4 summarizes the variables generated during the feature engineering process.

Table 4: Description of variables generated in the feature engineering process

Variable	Description
Vel_x^Y	Average speed (in km/h) of category x at gate Y .
$Vel_x^{Y,Z}$	Average speed between consecutive gates Y and Z for category x .
Sd_x^Y	Standard deviation of speed for category x at gate Y .
$Sd_x^{Y,Z}$	Standard deviation of speed between consecutive gates Y and Z for category x .
$Flow_x^Y$	Total number of vehicles of category x passing through gate Y during the interval.
$Flow_x^{Y,Z}$	Traffic volume between gates Y and Z for category x .
Den_x^Y	Vehicle density (vehicles per km) for category x at gate Y .
$Den_x^{Y,Z}$	Vehicle density between gates Y and Z for category x .
$\Delta Flow_x^Y$	Change in traffic volume between consecutive intervals for category x at gate Y .
$\Delta Flow_x^{Y,Z}$	Change in traffic volume between consecutive intervals across gates Y and Z for category x .
ΔVel_x^Y	Change in average speed for category x at gate Y .
ΔDen_x^Y	Change in vehicle density for category x at gate Y .
ΔSd_x^Y	Change in speed standard deviation for category x at gate Y .
$Ft_{Motorcycle}^Y$	Fraction of motorcycles in the total flow at gate Y .
Ft_{Heavy}^Y	Fraction of heavy vehicles in the total flow at gate Y .
$CL_x^{Y,Z}$	Proportion of vehicles in category x changing lane between gates Y and Z .
$\Delta Vel_x^{Y,Z}$	Change in spatial average speed between consecutive intervals for category x on segment (Y, Z) .
$\Delta Den_x^{Y,Z}$	Change in vehicle density between consecutive intervals for category x on segment (Y, Z) .
$Vel^{Y,Z}$	Average speed across all vehicle categories between gates Y and Z .
$Flow^{Y,Z}$	Total traffic volume across all vehicle categories between gates Y and Z .
$Sd^{Y,Z}$	Total speed standard deviation between gates Y and Z .
$Den^{Y,Z}$	Total vehicle density across all categories between gates Y and Z .
$\Delta Flow^{Y,Z}$	Change in total traffic volume between consecutive intervals on segment (Y, Z) .
$\Delta Vel^{Y,Z}$	Change in global average speed between consecutive intervals on segment (Y, Z) .
$\Delta Den^{Y,Z}$	Change in global vehicle density between consecutive intervals on segment (Y, Z) .
$Flow_\ell^{Y,Z}$	Traffic volume of vehicles originating from lane ℓ on segment (Y, Z) .
$Vel_\ell^{Y,Z}$	Average speed of vehicles originating from lane ℓ on segment (Y, Z) .
$Sd_\ell^{Y,Z}$	Speed standard deviation of vehicles from lane ℓ on segment (Y, Z) .
$Den_\ell^{Y,Z}$	Vehicle density from lane ℓ on segment (Y, Z) .
$\Delta Flow_\ell^{Y,Z}$	Change in flow for origin lane ℓ between consecutive intervals on segment (Y, Z) .
$\Delta Vel_\ell^{Y,Z}$	Change in average speed for origin lane ℓ between consecutive intervals on segment (Y, Z) .
$\Delta Den_\ell^{Y,Z}$	Change in density for origin lane ℓ between consecutive intervals on segment (Y, Z) .
$Acc(t)$	Binary indicator: 1 if at least one accident occurred during interval $t + 1$, 0 otherwise.

4.2. Data preparation

The literature has shown that the presence of incomplete or inconsistent records can introduce significant biases and negatively impact the performance of discriminative models (??). Therefore, a rigorous cleaning procedure was implemented, which included the removal of variables with a high percentage of missing data, the exclusion of intervals with incomplete observations, and the filtering of extended periods without recorded accidents potentially attributable to failures in the data capture infrastructure.

The data preparation process, presented in Figure 4, involved two critical stages to ensure the reliability of the dataset for predictive modeling. First, a rigorous treatment of missing data was applied. Variables with more than 1% of missing values were removed to avoid biased imputations or significant row loss. This led to the exclusion of **123 out of 260 predictive variables**, retaining a total of **137** predictive features. Additionally, all time intervals containing any missing value across the remaining variables were discarded, reducing the number of valid observations from **686,512 to 668,237**.

Secondly, a temporal filtering step was implemented to address sequences of several consecutive days with no recorded accidents. These anomalous windows, particularly those lasting seven days or more, raised concerns about potential failures in the data acquisition system, especially when coinciding with irregularities in AVI sensor activity. As a preventive measure, all affected periods were removed to avoid introducing artificial patterns into the models. In total, **30 anomalous windows were identified and 137,055 rows were excluded**. After applying both cleaning procedures, the final dataset comprised **531,182** five-minute intervals and **1,335** recorded accidents.

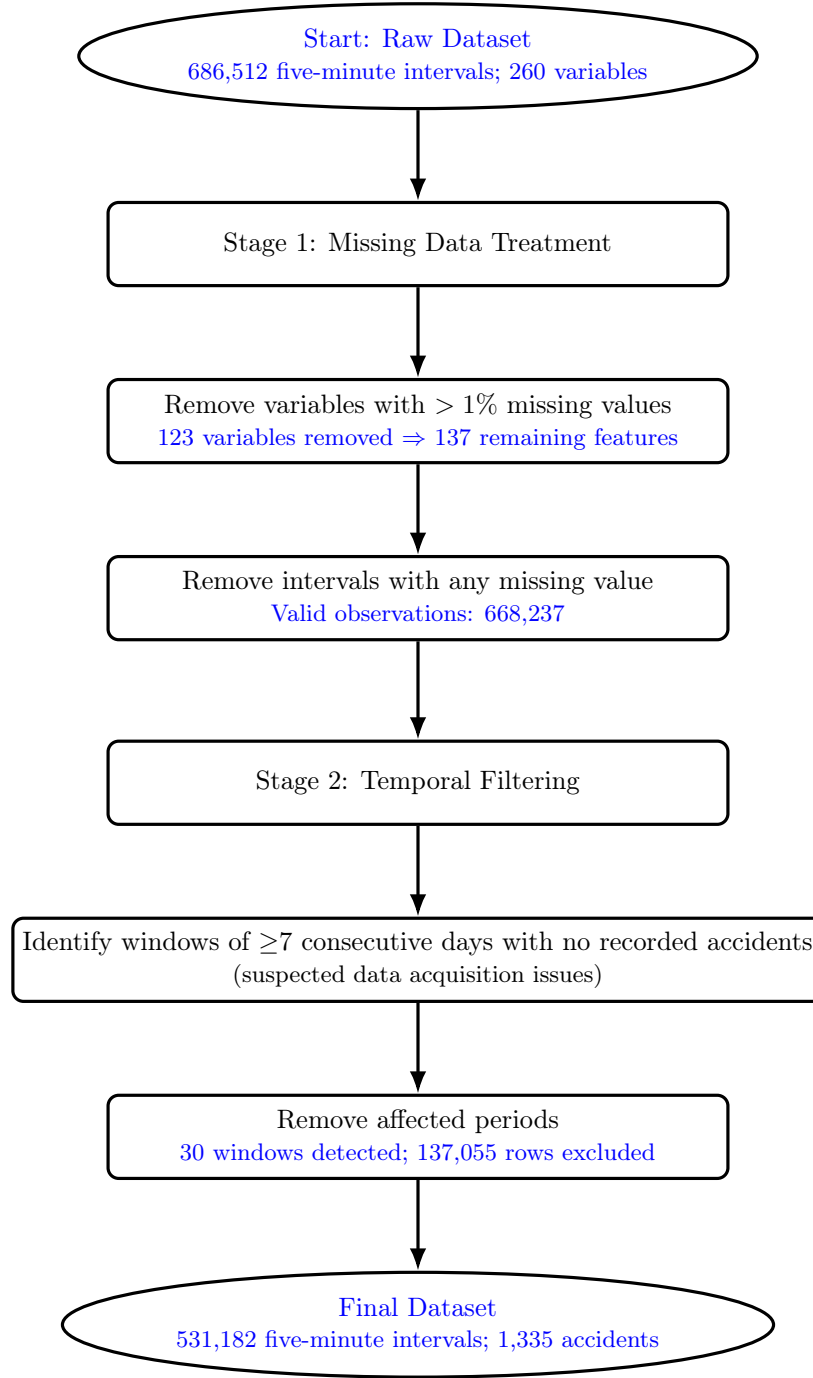


Figure 4: Data Preparation Flowchart. Stage 1 removes variables with over 1% missing data and then drops any intervals with missing values. Stage 2 filters anomalous multi-day windows with no recorded accidents, removing affected periods to avoid artificial patterns.

4.3. Variable selection

The variable selection process consists of two essential stages, which are carried out sequentially:

- Elimination of highly correlated variables.
- Variable selection using Random Forest.

In the following sections, experiments will be conducted in which the variable selection procedure must be standardized, in order to ensure a more rigorous analysis and allow for the replicability of results in later stages.

The variable selection process is carried out exclusively using data from the year 2018, which comprises 94,897 five-minute intervals and a total of 210 recorded accidents. The variables selected during this stage remain fixed throughout the entire study, ensuring methodological consistency and avoiding any data leakage from future periods into the model development process.

4.3.1. Correlation Matrix

The first stage of variable selection involves detecting and removing highly correlated variables in order to reduce redundancy and avoid multicollinearity issues. This analysis was applied exclusively to the training set corresponding to the year 2018, maintaining the independence of the test set and preventing data leakage.

Out of the 137 predictive variables remaining after filtering for missing data, pairwise collinearity was assessed using the Pearson correlation coefficient ρ_{xy} . This metric quantifies the linear relationship between two continuous variables. Given the high number of features, the procedure was automated, and variables with a correlation greater than 0.95 with at least one other variable in the set were removed. This threshold is commonly used in the literature to identify strong relationships that do not add additional value to the model and may induce overfitting or instability in parameter estimation (?). Eliminating these variables improves model interpretability and significantly reduces dimensionality without substantial loss of information.

This step not only enhances model interpretability but also reduces dimensionality and mitigates the risk of overfitting during training. A total of 38 highly correlated variables with a coefficient $\rho_{xy} > 0.95$ relative to other variables in the set were removed leaving us with 166 variables with no high correlation. Figure 5 shows the density distribution of the absolute values of the pairwise correlation coefficients ($|\rho_{ij}|$) among all predictive variables in the training set. Most variable pairs exhibit very low correlation, while only a small proportion exceed the cutoff of 0.95 (indicated by the red dashed line). The cutoff selected is based on ?. This justifies the choice of the threshold and illustrates that the majority of variables provide complementary information, with only a limited number of highly redundant features that were systematically excluded from further analysis.

Figure unavailable in this workspace
Imagenes/Density.png

Figure 5: Density of the absolute values of pairwise correlation coefficients ($|\rho_{ij}|$) among predictive variables. The vertical red dashed line indicates the exclusion threshold ($|\rho| = 0.95$).

4.3.2. Random Forest for feature selection

After eliminating highly correlated variables, an additional feature selection step was carried out using a Random Forest model, an approach widely recognized for its ability to identify relevant predictive variables in nonlinear contexts with complex interactions (????).

The Random Forest algorithm operates by constructing an ensemble of decision trees using bootstrap samples of the original dataset. At each split in a tree, a random subset of predictors is evaluated to determine the optimal split point, using the Gini impurity as the criterion. This mechanism not only enhances the diversity among trees but also provides a robust framework for assessing variable importance. Specifically, the importance of each predictor is quantified by the average decrease in Gini impurity across all trees when that variable is used for splitting. Furthermore, the model’s generalization capacity is evaluated using Out-of-Bag (OOB) samples, those not included in each bootstrap iteration, allowing performance metrics such as accuracy and AUC to be estimated without the need for a separate validation set.

The final *Random Forest* model was trained on a total of 75,917 observations from 2018, using 166 predictive variables, and performing binary classification between intervals with and without accidents. During the calibration process, different values of the complexity parameter *mtry* were tested, with the best performance obtained at *mtry* = 84, achieving an average AUC of 0.584, perfect sensitivity (1.00), and zero specificity (0.00). Although this combination reveals a strong tendency of the model to predict the majority class, our primary objective was to use this structure to estimate the relative importance of each predictor. This estimation was based on the average decrease in impurity between classes, which enabled the generation of a detailed ranking of predictor variable relevance.

Based on this ranking, the 15 most important variables were selected for the accident prediction task, prioritizing those with the highest discriminative power. Among these are, for example, spatial differences in heavy vehicle flow ($\Delta\text{Flow.15.14.Heavy}$), lane-specific speed variations ($\Delta\text{Vel.15.14.Lane2}$), absolute car speed (Vel.Cars.14), and speed dispersion ($\Delta\text{SD.Cars.14}$). Figure 6 shows the top 20 variables with the highest relative importance, highlighting the central role of temporal, spatial, and disaggregated dimensions by vehicle type or lane in the model’s predictive capacity.

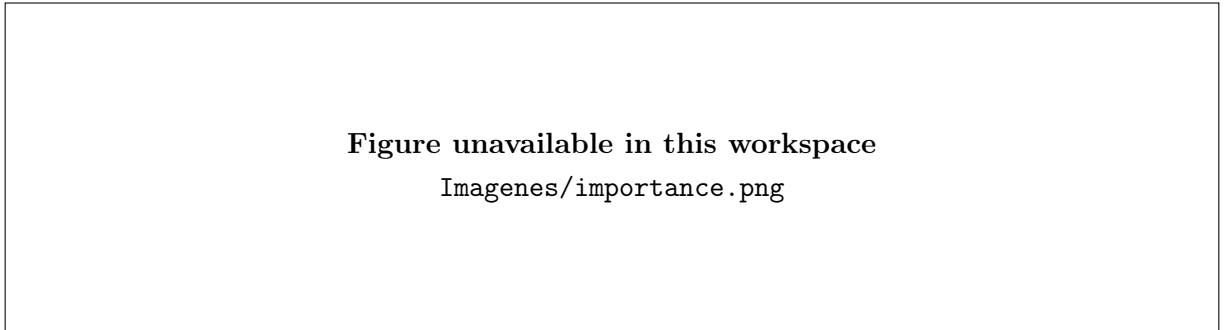


Figure 6: Importance ranking of the 20 most relevant variables according to Random Forest

4.4. Drift

In the context of machine learning and predictive modeling, the term drift refers to the variability or change in data distributions over time (?). This phenomenon can compromise a model’s predictive capability, as the patterns observed during training may become obsolete as underlying conditions change. Broadly speaking, there are two main types of drift: data drift and concept drift. To address these challenges, several drift detection algorithms have been proposed, with ADWIN being one of the most widely adopted due to its theoretical guarantees and ability to adjust dynamically to changes without requiring parameter tuning for the window size. Comparative studies such as those by ?? have highlighted ADWIN’s consistent performance across multiple scenarios, particularly in streaming and highly imbalanced environments. Furthermore, ? emphasizes its utility as a lightweight and efficient component within ensemble frameworks for non-stationary data, making it a suitable candidate for applications like ours where real-time recalibration and computational feasibility are critical.

Data drift (also known as *covariate shift*) occurs when the distributions of input predictors (explanatory variables) change over time, while the relationship between these attributes and the target variable (dependent variable) remains constant. In other words, the model continues to map inputs to outputs in the same way, but the nature of the inputs themselves evolves, diverging from the distributions observed during the training phase. This phenomenon often leads to a degradation in model performance, as the system is no longer exposed to familiar patterns. Data drift is particularly prevalent in real-time systems that process continuous data, which is often affected by dynamic contextual conditions such as traffic or weather. These systems are particularly susceptible to variations in inputs due to seasonality, infrastructure changes, or external disruptions. The first studies in the field of *drift* emerged as machine-learning-based decision support systems began to be deployed in these types of non-stationary environments. Notably, the works of ? and ? were pioneering in recognizing drift as a critical phenomenon and proposing adaptive mechanisms for maintaining model relevance in changing data conditions.

Concept drift, on the other hand, refers to a change in the relationship between input predictors and the target variable (?). In other words, the model can no longer make accurate predictions because the concept or nature of the output data has changed. This type of drift is significantly more problematic, as not only have the input data changed, but also the underlying structure that links the predictors with the target variable of interest (?).

The main difference between *data drift* and *concept drift* lies in which component of the data distribution changes over time. *Data drift* refers to changes in the marginal distribution of the input features, that is, $P(X)$, without altering the conditional relationship between these features and the target variable. This occurs, for example, when the types or frequencies of inputs the model receives change, such as shifts in traffic volume, weather patterns, or road usage, while the underlying relationship between inputs and outcomes remains stable. Since $P(X)$ captures how the input data is distributed, any variation in context, behavior, or environment that modifies the frequency or nature of inputs constitutes data drift. In contrast, *concept drift* refers to changes in the conditional distribution $P(Y | X)$, meaning that even if the inputs remain the same, the way they relate to the output changes over time. This reflects an evolution in the underlying data-generating process that governs the target variable. In the context of crash prediction, for instance, concept drift may arise when external factors, such as changes in driver behavior, enforcement policies, or road infrastructure, alter the risk dynamics associated with specific traffic patterns. Because $P(Y | X)$ directly defines the model’s decision function, concept drift represents a more fundamental shift that cannot be corrected by reweighting inputs alone and typically requires retraining or adaptive learning mechanisms (????).

To address the phenomenon of concept drift in online data environments, several strategies have been proposed, among which the *ADWIN* (Adaptive Windowing) algorithm introduced by ? stands out. This algorithm enables the real-time detection of concept changes by continuously monitoring the mean within an adaptive sliding window. As new data is incorporated, *ADWIN* evaluates potential partitions of the current window and automatically reduces its size if statistically significant differences are detected between the subsets, indicating a change in the distribution. The detailed procedure of this approach is summarized in Algorithm Algorithm 1.

To detect a significant change between two consecutive sub-windows W_0 and W_1 , the difference between their empirical means is evaluated:

$$|\hat{\mu}_{W_0} - \hat{\mu}_{W_1}| > \varepsilon_{\text{cut}}$$

where $\hat{\mu}_{W_0}$ and $\hat{\mu}_{W_1}$ are the means of windows W_0 and W_1 , respectively. The detection threshold ε_{cut} is computed using a Hoeffding bound adjusted to the size of the windows:

Algorithm 1 Adaptive Windowing Algorithm (ADWIN) (?)

1. Initialize Window W
 2. **for each** $t > 0$
 3. **do** $W \leftarrow W \cup \{x_t\}$ (i.e., add x_t to the head of W)
 4. **repeat** Drop elements from the tail of W
 5. **until** $|\hat{\mu}_{W_0} - \hat{\mu}_{W_1}| \geq \varepsilon_{\text{cut}}$ holds
 6. for every split of W into $W = W_0 \cdot W_1$
 7. **output** $\hat{\mu}_W$
-

$$\varepsilon_{\text{cut}} = \sqrt{\frac{1}{2m} \cdot \ln\left(\frac{4n}{\delta}\right)}$$

where:

- $m = \frac{1}{\frac{1}{|W_0|} + \frac{1}{|W_1|}}$ is the harmonic mean of the sizes of the subwindows W_0 and W_1 ,
- $n = |W|$ is the total size of the current window $W = W_0 \cup W_1$,
- δ is the statistical significance level used to control the false positive rate.

To address real-time *concept drift* detection, the *ADWIN* algorithm was selected due to its unique ability to automatically adjust the analysis window size based on statistical evidence of changes in the data distribution. This approach enables the detection of significant transitions in the relationship between explanatory variables and the target variable without the need to store the entire data history. Upon identifying a *drift* point, *ADWIN* discards the oldest data (window W_0) and retains only the most recent subwindow W_1 , on which the model can be retrained to remain up to date with changing conditions.

4.5. Prediction Models

In this section, we describe the models that are used in the next experiments

XGBoost: Extreme Gradient Boosting (XGBoost), developed by ?, is a decision tree-based model that learns sequentially, improving with each iteration by focusing on the errors made in previous iterations. Its design makes it particularly efficient and accurate, and it is widely used in data science due to its ability to handle nonlinear relationships and large datasets, with several successful applications in the transportation domain (?).

AdaBoost: Adaptive Boosting (AdaBoost) (?) combines multiple simple models, known as weak learners, into a strong classifier. In each round, it shifts attention to the errors made by the previous model, assigning more weight to the misclassified observations.

Random Forest: Random Forest (?) builds multiple decision trees using random subsets of data and variables, then combines their predictions to make a final decision. This approach helps reduce overfitting and improves the model's generalization ability.

Neural Network (NNet): Neural networks (?) are models inspired by the human brain. Through layers of interconnected nodes, they can learn complex patterns in data. In this study, a simple multilayer neural network is used to classify binary outcomes, leveraging its ability to model nonlinear relationships among variables. Prior to training, all predictors are standardized using a [StandardScaler](#) fitted exclusively on the training partition of each fold (median imputation is applied first); the resulting transform is then applied to validation, test, and stream data using the same parameters, preventing leakage across

folds and across time. The optimizer is set to `adam` with early stopping (patience of 20 epochs, tolerance 10^{-4}) and a 10% internal validation fraction.

The decision threshold was not fixed at the conventional value of 0.5, but was instead dynamically adjusted for each model-year combination through an internal validation process. Specifically, in each training instance, 20% of the training data was set aside as a stratified validation set. On this partition, the predicted probabilities from the model were computed, and the corresponding ROC curve was constructed. From this curve, the optimal decision threshold was identified using the Youden criterion (?), defined as:

$$\theta^* = \arg \max_{\theta} (\text{Sensitivity}(\theta) + \text{Specificity}(\theta) - 1)$$

This threshold simultaneously maximizes sensitivity (recall) and specificity, allowing for an appropriate trade-off between true positives and true negatives. This strategy is particularly relevant in classification problems with highly imbalanced classes, such as accident prediction, where the positive class (accidents) represents a tiny fraction of the total observations. Dynamic threshold calibration ensures that model comparisons are not biased by an arbitrary cutoff choice, and that each model operates under conditions optimized concerning its actual discriminative power (?).

Hyperparameter tuning was performed using Bayesian optimization via Optuna (?) with a Tree-structured Parzen Estimator (TPE) sampler. The objective function maximizes the stratified 5-fold cross-validated AUC computed on the inner training partition T^{fit} . Formally, for model m and balance mode b ,

$$\hat{\theta} = \arg \max_{\theta \in \Theta_{m,b}} \frac{1}{K} \sum_{k=1}^K \text{AUC}_k(\theta),$$

where $K = 5$ and each fold is stratified to preserve the crash/non-crash ratio. The number of Optuna trials is capped at the size of the discrete search space for each model, up to a maximum of 30 trials in full mode. To guarantee comparability across recalibration strategies, tuning is performed *once* on the base year Y_0 for each (model, balance_mode, seed) combination, and the resulting canonical $\hat{\theta}$ is reused by all downstream yearly and adaptive blocks.

Table 5 presents the discrete search spaces used. For **Random Forest**, the search covers the number of candidate features at each split (`mtry`), the splitting criterion (`gini` or `extratrees`), and the minimum leaf size (`min_node_size`). When `balance_mode = none`, `class_weight = "balanced_subsample"` is set to compensate for class imbalance; this option is disabled when SMOTE is active. For **XGBoost**, the search covers tree depth (`max_depth`), learning rate (`eta`), minimum loss reduction (`gamma`), column and row subsampling fractions (`colsample_bytree`, `subsample`), and the minimum instance weight per leaf (`min_child_weight`), with a fixed number of boosting rounds (`nrounds = 100`). For **AdaBoost**, the search varies the number of boosting iterations (`iter`), the depth of the weak learner (`maxdepth`), and the shrinkage rate (`nu`). For the **Neural Network**, the number of neurons in the single hidden layer (`size`), the L2 weight-decay regularization (`decay`), and the maximum training iterations (`maxit`) are searched; the network is trained with the `adam` optimizer, early stopping with patience of 20 epochs and tolerance 10^{-4} , and a 10% internal validation fraction.

When `balance_mode = smote`, SMOTE parameters (`sampling_strategy` $\in \{0.3, 0.5, 0.75, 1.0\}$ and `k_neighbors` $\in \{3, 5, 7\}$) are jointly optimized as part of the same Optuna study. SMOTE is applied exclusively to training folds and to the final refit; the test year and evaluation stream are never resampled. Once $\hat{\theta}$ is fixed, a final model is refit on all rows of the full training set T , optionally augmented via SMOTE. The operating threshold is calibrated using the Youden criterion applied to out-of-fold predicted scores generated during cross-validation on T ; if this is ill-posed (e.g., only one class in the calibration set), the code falls back to a stratified 20% internal holdout.

Table 5: Discrete search spaces evaluated by Optuna (TPE sampler) for each model. When `balance_mode = smote`, `sampling_strategy` $\in \{0.3, 0.5, 0.75, 1.0\}$ and `k_neighbors` $\in \{3, 5, 7\}$ are added to each model’s search space.

Model	Search space
Random Forest	<code>mtry</code> : {2, 4, 6} <code>splitrule</code> : {gini, extratrees} <code>min_node_size</code> : {1, 5, 10} Search space size: 18 combinations; up to 18 Optuna trials. <code>class_weight = balanced_subsample</code> when <code>balance_mode = none</code> .
XGBoost	<code>max_depth</code> : {3, 6, 9} <code>eta</code> : {0.01, 0.1, 0.3} <code>gamma</code> : {0, 1, 5} <code>colsample_bytree</code> : {0.5, 0.8, 1.0} <code>min_child_weight</code> : {1, 5, 10} <code>subsample</code> : {0.5, 0.7, 1.0} <code>nrounds</code> : 100 (fixed) Search space size: 729 combinations; up to 30 Optuna trials.
AdaBoost	<code>iter</code> : {50, 100, 150} <code>maxdepth</code> : {1, 2, 3} <code>nu</code> : {0.1, 0.5, 1.0} Search space size: 27 combinations; up to 27 Optuna trials.
Neural Network (NNet)	<code>size</code> : {6, 7, 10, 12, 13, 14, 15, 16} <code>decay</code> : {0.095, 0.1, 0.15, 0.2} <code>maxit</code> : {300, 500, 800} Search space size: 96 combinations; up to 30 Optuna trials. Optimizer: adam; early stopping: patience 20, tol 10^{-4} , val. fraction 0.10. Non-converged folds treated as invalid; trials returning AUC < 0 are rejected.

4.6. Performance Indicators

Performance evaluation was conducted on an annual basis, generating predictions for each test year and comparing the results with the actual accident occurrence labels. The metrics used include:

- **Area Under the ROC Curve (AUC):** This metric evaluates the model’s ability to distinguish between classes by plotting the true positive rate against the false positive rate at various decision thresholds. It does not have a closed-form formula, as it is computed as the area under the ROC curve, with values ranging from 0.5 (random guessing) to 1 (perfect classifier).
- **Sensitivity (Recall):** Measures the model’s ability to identify all actual positives correctly.

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

where TP stands for true positives and FN for false negatives.

- **Specificity (True negative rate):** This metric measures the proportion of actual negative cases that are correctly identified as negative by the model, representing the model’s ability to avoid false positive predictions:

$$\text{Specificity} = \frac{TN}{TN + FP}$$

where TN stands for true negatives and FP for false positives.

- **Error Rate:** This metric quantifies the overall proportion of incorrect predictions made by the model in all classes, providing a comprehensive measure of the accuracy of the model:

$$\text{Error Rate} = \frac{FP + FN}{TP + TN + FP + FN}$$

4.7. Experimental design

The experimental design of this study aims to evaluate the generalization capability and robustness of various binary classification models for predicting the occurrence of accidents in 5-minute intervals.

- **No recalibration:** In this scenario, a single model is trained using data from the year 2018 and is applied without any adjustments to predict each of the subsequent years (2019–2024). This approach allows for the evaluation of the model’s generalization ability in the face of temporal changes without any updates.
- **Recalibration using the previous year (*period-aligned retraining*):** For each evaluation year t , a new model is trained using only the data from year $t - 1$. This strategy captures recent traffic and accident dynamics, annually recalibrating the model with the most recent available information.
- **Cumulative recalibration:** A model is trained using all available data from 2018 up to the year immediately before prediction ($\leq t-1$). This approach uses the accumulation of historical experience, allowing the model to incorporate long-term trends.
- **Adaptive retraining with *concept drift* detection (ADWIN):** In this experiment, the *ADWIN* algorithm is used to detect changes in the distribution of prediction errors in real time, applying an automatic recalibration approach. The procedure starts with training a base model on 2018 data. Then, sequential prediction is performed data point by data point (5 minutes period) for the period following 2018. After each prediction, it is checked whether the model made an error (1 if incorrect, 0 if correct), and this binary indicator is passed to the ADWIN algorithm. If no change is detected in the error distribution, predictions continue using the current model. If *drift* is detected, the change point is recorded, relevant metrics are computed, and the model is retrained using only the last $|W_1|$ stream rows identified by ADWIN. Retraining is triggered only if the W_1 window contains at least $\max(20, \lfloor W_{\min}/10 \rfloor)$ rows and both target classes are present; otherwise the current model continues unchanged. This procedure enables continuous and automatic adaptation to conceptual changes in the data, without requiring explicit definition of recalibration instances. In our implementation, we set the ADWIN confidence parameter to $\delta = 0.002$, the value recommended by ? as a good balance between false-alarm rate and drift sensitivity, and required a minimum window size of $W_{\min} = 45,000$ observations so that every retraining step would draw on a sufficiently large sample to stabilise gradient-based learners in this highly imbalanced, non-stationary setting.
- **Adaptive Random Forest (ARF):** This strategy implements a genuinely online learning approach. An initial ARF ensemble is trained and threshold-calibrated on the base-year data Y_0 using a trailing chronological validation block. During the subsequent stream, the model is updated one observation at a time:

$$s_t = f_{t-1}(x_t), \quad f_t = \mathcal{U}(f_{t-1}, x_t, y_t),$$

where $\mathcal{U}$ denotes the online `learn_one` update of the *river* library (?). This formulation is consistent with Adaptive Random Forest as an ensemble for evolving data stream classification, designed to update incrementally as new observations arrive (?). Unlike ADWIN and KSWIN, ARF handles

drift internally through its forest mechanism without external batch retraining. By contrast, standard SMOTE is a batch resampling procedure that synthesizes minority observations from nearest neighbors computed within an available training set (?). Because the ARF pipeline consumes the stream one observation at a time rather than reassembling a batch training set before each update, that standard batch-level resampling step is not available here. Stream-specific oversampling variants have been proposed precisely for evolving data streams (?), but they are outside the scope of the present ARF implementation. Consequently, ARF is evaluated only under the **none** (no balancing) configuration. Results are reported by prediction year.

- **Adaptive recalibration with KSWIN:** The Kolmogorov–Smirnov Windowing (KSWIN) detector (?) monitors covariate drift on a selected subset of the top- k features, rather than the prediction error stream used by ADWIN. For each monitored feature f , drift is assessed either on the raw value $x_{t,f}$ or on a seasonal z-score

$$z_{t,f} = \frac{x_{t,f} - \mu_{d(t),h(t),f}}{\sigma_{d(t),h(t),f}},$$

with fallback to hour-level and global base-year profiles. Each detector uses $\alpha = 10^{-4}$, **window_size**=300, and **stat_size**=30 (fixed window sizes $W = 300$, $W_0 = 270$, $W_1 = 30$ in Appendix A.9). A drift event is declared when the number of simultaneously firing detectors exceeds a configurable vote threshold. After detection, the model is retrained on a temporal window of the last D days; if that window is too small or lacks both classes, it expands to all observations seen up to t .

Each of these strategies was applied to four classification algorithms widely used in the accident prediction literature: XGBoost, AdaBoost, Random Forest and neural networks (NNet). For all experimental strategies, the same training and validation procedures were applied, based on 5-fold stratified cross-validation. To address the severe class imbalance (crash events represent less than 0.2% of observations), experiments are run under two balance modes: **none** (no resampling) and **smote**, in which Synthetic Minority Over-sampling Technique (SMOTE) (?) is applied exclusively to training folds and the final refit set; the test data and evaluation stream are never resampled. In its original form, SMOTE is a batch resampling method that generates synthetic minority samples from nearest neighbors in the available training set, which is compatible with the batch-trained strategies evaluated here. SMOTE sampling strategy and neighborhood size k are included in the Optuna search space when active. The sole exception is the ARF strategy, which follows an online data stream learning paradigm (??): observations are processed one at a time, so there is no batch training set on which standard SMOTE can be applied. Although stream-oriented oversampling methods exist (?), they were not part of the ARF pipeline considered in this study; accordingly, ARF is evaluated only under the **none** mode.

5. Experimental Results

This section presents the results obtained from the implementation of the experimental setting described in Subsection 4.7. The analysis is organized according to each proposed recalibration strategy, starting with the scenario without any adjustment after the initial training (static model), and progressing toward periodic, cumulative, and adaptive recalibration methodologies.

The results are reported annually for the 2019 to 2024 period, allowing for an evaluation of each model’s predictive performance over time. The performance indicators used for comparison are the ones described in detail in Subsection 4.6. The training set consisted of the 15 most relevant variables according to feature importance. All experiments were conducted on an Intel Core i9-14900K CPU (32 logical processors,

128GB RAM) running R version 4.4.1. Each experiment was repeated three times with different random seeds to ensure reproducibility, and reported metrics represent the mean values.

All computational results are detailed in Appendix A (Tables A.6-A.9), with visual summaries presented through the averaged ROC curves in Figure 7. These curves represent key insights into how model performance changes across different recalibration strategies.

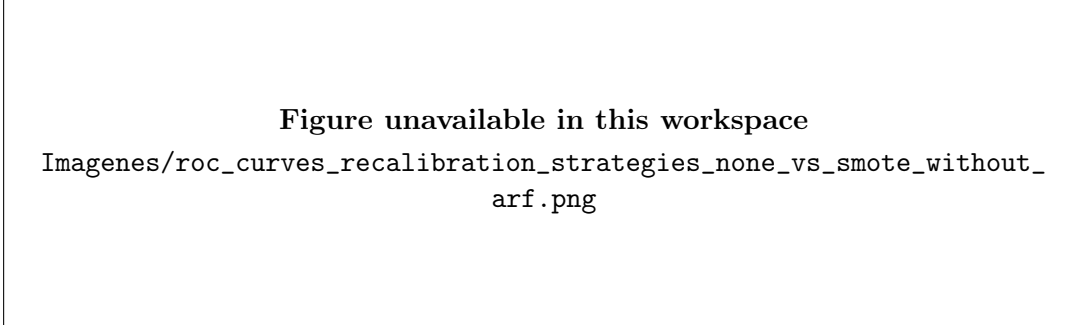


Figure 7: Average ROC curves for each recalibration strategy by model

Overall, the cumulative strategy delivers the best AUC values across most models. XGBoost and AdaBoost achieve their highest performance under this approach (average AUC of 0.798 and 0.778, respectively, without class balancing; Random Forest also reaches 0.796 under cumulative training), with XGBoost reaching a peak single-year AUC of 0.866 in 2023 (sensitivity = 0.801), outperforming both static and all adaptive strategies (ADWIN, KSWIN, and ARF). A critical finding is the role of SMOTE-based oversampling: for NNet, SMOTE raises the average cumulative AUC from 0.517 to 0.773, transforming a consistently underperforming model into one competitive with ensemble methods. In contrast, XGBoost and AdaBoost show marginal differences with and without SMOTE, indicating that their boosting mechanisms already handle class imbalance effectively. These results confirm that aggregating historical data improves model robustness and predictive accuracy, especially for ensemble models, and that appropriate class balancing can substantially improve neural network performance in rare-event prediction.

Additional patterns from the ROC curves reinforce these conclusions. The cumulative recalibration strategy consistently yields the highest AUCs, confirming the benefit of long-term data aggregation. In contrast, the period-aligned recalibration often under-performs and, in some cases, performs worse than the static baseline. This is particularly evident during periods of structural change such as the COVID-19 pandemic, when training on the previous year (e.g., 2019 for 2020 predictions) leads to marked declines in sensitivity for XGBoost and AdaBoost. The lowest AUCs for both models under this strategy occur in 2021, when models were retrained on 2020 data.

Although designed to respond dynamically to data changes, the three adaptive strategies exhibit distinct failure modes. ADWIN fragments training data through frequent drift detections (up to 28 segments for XGBoost with SMOTE and 26 for Random Forest with SMOTE), yielding an average AUC of 0.723 for XGBoost without balancing and highly unstable results for NNet without SMOTE (per-segment AUC ranging from 0.22 to 0.60). KSWIN, which monitors feature distributions rather than prediction error, over-fragments the data stream—generating 74 segments across the 2018–2024 period, of which only 5 produce enough observations for valid evaluation. This excessive segmentation degrades performance well below the static baseline (e.g., average AUC of 0.673 for XGBoost vs. 0.779 static), yielding no genuine adaptive benefit. Most strikingly, ARF proves entirely inadequate for this rare-event context: all three voting configurations (fast, majority, moderate) achieve AUCs below 0.48 and near-zero sensitivity, essentially predicting all observations as the majority class. None of these adaptive approaches compensates for the gains offered by cumulative learning, confirming that rare-event crash prediction

fundamentally favors data accumulation over adaptive windowing.

We now dig further into the performance of each of the four predictive models.

- **XGBoost** remains the top-performing model overall, particularly under cumulative strategies. Its AUC scores frequently exceed 0.75, reaching a peak of 0.866 in 2023 with a sensitivity of 0.801, while sensitivity improves with larger datasets. These gains, however, come at the cost of increasing computational time as the training set grows (Table A.8). Under adaptive strategies, XGBoost shows mixed results: KSWIN (average AUC = 0.673) performs substantially below the static baseline (0.779) due to excessive data fragmentation from 74 detected drift segments, while ADWIN also reduces the average AUC to 0.723. SMOTE provides negligible improvement for XGBoost (cumulative AUC: 0.795 vs. 0.798 without balancing), suggesting that the boosting mechanism already handles imbalance effectively.
- **AdaBoost** offers a strong balance between accuracy and efficiency, achieving AUCs comparable to XGBoost with substantially lower computational cost. It performs best under the cumulative strategy (average AUC = 0.778) and shows consistent performance across non-adaptive strategies: its static AUC (0.761) is only marginally lower than cumulative. Under ADWIN, performance degrades in both configurations (average AUC of 0.701 without SMOTE and 0.676 with SMOTE), with the algorithm generating up to 25 drift segments without balancing that fragment the training data excessively. KSWIN performs even worse (average AUC = 0.649 without SMOTE), again due to over-fragmentation of the data stream (Table A.10).
- **Random Forest** emerges as a strong competitor to XGBoost, achieving the highest cumulative AUC among all models without balancing (average AUC = 0.796), with a peak of 0.839 in 2023. Under the static strategy, it also performs well (average AUC = 0.783 without SMOTE, 0.788 with SMOTE), reaching 0.847 with SMOTE in 2023—comparable to XGBoost’s peak of 0.866. Interestingly, Random Forest with SMOTE achieves a slightly higher cumulative AUC (0.797) than without (0.796), though this difference is negligible, confirming that bagging-based ensembles inherently handle class imbalance. Period-aligned retraining yields a modest decline (AUC = 0.780), while ADWIN-based adaptation produces sharp degradation: average AUC of 0.752 across 12 drift segments without SMOTE and 0.745 across 26 segments with SMOTE, consistent with the excessive data fragmentation observed for other models. KSWIN performs even worse (average AUC = 0.682 without SMOTE), as its 74 detected segments over-fragment the training data. Overall, Random Forest’s consistent performance across non-adaptive strategies, combined with moderate training times, makes it a viable alternative to XGBoost when computational simplicity is preferred.
- **NNet** exhibits the most dramatic response to class balancing. Without SMOTE, NNet is the weakest model across all strategies, with average AUCs between 0.459 (KSWIN) and 0.517 (cumulative), consistently overfitting to the majority class. With SMOTE, however, NNet undergoes a transformative improvement: cumulative AUC rises to 0.773 and reaches a peak of 0.831 in 2023—approaching the performance of ensemble methods (AdaBoost achieves 0.835 in that same year). Under adaptive strategies, NNet with SMOTE achieves 0.565 (KSWIN) and 0.668 (ADWIN), where the high number of drift segments (up to 17 for ADWIN, 74 for KSWIN) fragments training data and destabilizes learning. This sensitivity to class balancing suggests that NNet’s previous poor performance was driven primarily by imbalance rather than inherent model capacity limitations.

While the adaptive strategies offer theoretical advantages for handling concept drift, their effectiveness drops in highly imbalanced contexts like crash prediction, where the positive class accounts for only 0.2%

of the data. Each method exhibits a distinct failure mode: ADWIN produces excessive drift signals (up to 28 segments for XGBoost with SMOTE, and 26 for Random Forest with SMOTE) driven by fluctuations in the majority class, fragmenting training data and preventing models from learning long-term patterns; KSWIN, monitoring feature distributions rather than prediction error, over-fragments the data even more severely—detecting 74 segments across the 2018–2024 period, of which only 5 yield valid evaluations—resulting in average AUCs substantially below the static baseline (e.g., 0.673 for XGBoost vs. 0.779 static) and offering no adaptive advantage; and ARF, designed for online learning on streaming data, proves entirely unable to learn meaningful patterns from the extreme class imbalance ($\text{AUC} < 0.48$ across all configurations), with its majority and moderate voting variants producing identical results. These limitations point to the need for domain-specific drift detection methods better aligned with rare-event traffic safety applications, where the base rate of the positive class fundamentally challenges standard drift detection assumptions. Similar shortcomings of ADWIN have also been observed in other areas, such as document classification (?).

6. Concluding Remarks

Accurate crash prediction is essential for improving road safety, yet maintaining model performance over time remains a challenge due to changing traffic conditions and data distributions. Previous research has advanced a variety of predictive models, from statistical and machine learning approaches to modern deep learning methods, and has explored recalibration techniques to counter performance degradation. However, most studies have focused on single-year evaluations or limited update strategies, leaving open questions about how different recalibration methods perform in real-world, multi-year contexts. To address this gap, we systematically evaluated six recalibration strategies: static, period-aligned, cumulative, and three adaptive approaches (ADWIN, ARF, and KSWIN); applied to RTCP models (XGBoost, AdaBoost, Random Forest, and NNet) using vehicle-level data from the Autopista Central in Santiago, Chile. Class imbalance was addressed through SMOTE-based oversampling, and hyperparameters were tuned via Bayesian optimization with Optuna. Our analysis compares these strategies across multiple years, providing empirical evidence on their impact on predictive accuracy, sensitivity, and computational efficiency in a dynamic traffic environment.

The findings indicate that ensemble-based methods, particularly XGBoost and AdaBoost, consistently achieved the highest predictive performance. Of these two methods, XGBoost achieved superior overall accuracy and sensitivity, while AdaBoost provided comparable predictive performance with shorter training times. The cumulative strategy proved most effective overall, consistently delivering the highest AUC values (0.798 for XGBoost, 0.796 for Random Forest, and 0.778 for AdaBoost without balancing) by leveraging historical data to capture long-term traffic patterns. In contrast, all three adaptive recalibration approaches underperformed: ADWIN was hindered by excessive and unstable drift detections (up to 28 segments), KSWIN over-fragmented the data into 74 segments—most too small for valid evaluation—yielding AUCs well below the static baseline, and ARF proved entirely inadequate for rare-event prediction ($\text{AUC} < 0.48$). A key new finding is that SMOTE-based oversampling transforms NNet from the weakest model (cumulative $\text{AUC} = 0.517$ without balancing) into one competitive with ensemble methods ($\text{AUC} = 0.773$), indicating that class imbalance—not model capacity—was the primary driver of NNet’s poor performance in prior evaluations.

A key finding is the sharp performance drop in some cases when models were recalibrated with only the preceding year’s data, particularly during the COVID-19 pandemic. Training on 2019 data to predict 2020 resulted in AUC drops due to the model’s inability to capture pandemic-induced traffic changes, including lockdowns and altered commuting patterns, affected commuting patterns, and fluctuations in vehicle flow. This led to poor generalization, reduced accuracy, and unstable predictions, showing the risk of short-term, period-aligned retraining in volatile contexts. In contrast, cumulative recalibration, while

more computationally demanding, used long-term historical data to capture broader trends, producing more stable and accurate results across changing traffic conditions.

Another important result is that NNet’s performance is highly sensitive to class balancing: without SMOTE, it consistently overfits to the majority class across all strategies, but with SMOTE it achieves a cumulative peak AUC of 0.831 in 2023, approaching AdaBoost’s 0.835 in that same year. This finding has practical implications for operational RTCP systems, as it demonstrates that the choice of class balancing technique can be as consequential as model selection. Random Forest proves surprisingly competitive across all strategies, achieving the highest cumulative AUC without balancing (0.796) and approaching XGBoost’s peak performance in 2023 (AUC = 0.847 static with SMOTE, 0.839 cumulative without SMOTE). Under ADWIN, however, it suffers from substantial data fragmentation (up to 26 segments with SMOTE), while XGBoost with SMOTE reaches the highest segment count of 28, confirming the instability of frequency-based drift detection in this context. Overall, the results point to ensemble methods such as XGBoost and AdaBoost as the most stable and adaptable, with fewer high-confidence errors across strategies.

Taken together, these findings provide practical guidelines for the operation of RTCP systems. Among them, cumulative recalibration stands out as the most robust and accurate strategy, as it uses historical information to maintain model stability and outperform both static schemes and period-aligned recalibration, the latter proving fragile in the face of abrupt structural changes (e.g., COVID-19). Furthermore, the results suggest that the use of drift detectors such as ADWIN, KSWIN, and ARF should not be aimed at replacing cumulative recalibration, but rather at complementing it in operational practice: their greatest value lies in monitoring online error and triggering review or retraining alerts under persistent conditions of change, thereby functioning as an adaptive surveillance mechanism. The failure of ARF in this context is particularly noteworthy: despite its success in other streaming classification tasks, its inability to handle extreme class imbalance (0.2% positive rate) renders it ineffective for rare-event safety applications without substantial algorithmic adaptation.

Some limitations should be acknowledged. This study examined six recalibration strategies (static, period-aligned, cumulative, and three adaptive approaches: ADWIN, ARF, and KSWIN). Other modeling approaches, update methods, and concept drift detection algorithms were not explored and may perform better in some settings. The results also reflect a specific case: crash prediction on Chile’s Autopista Central, a tolled urban highway with free-flow traffic. Infrastructure characteristics, local driving behavior, and seasonal patterns, including national holidays, may differ elsewhere, so the generalization of these findings to other regions or traffic systems should be made with caution.

This study points to several next steps. One is to test recalibration methods that take into account traffic cycles and seasons, since patterns often repeat by time of day, day of week, or month. Training models with data from the same calendar period could improve accuracy, especially in Chile where crashes peak every September during national holidays. Another step is to design drift detection and recalibration methods better suited to local traffic instead of only using general tools like ADWIN, KSWIN, or ARF. The dramatic impact of SMOTE on NNet performance also motivates further investigation into class balancing strategies tailored to rare-event prediction, including cost-sensitive learning and adaptive oversampling techniques that account for evolving class distributions. Finally, combining adaptive drift detection with cumulative retraining may help balance quick reactions to changes with learning from long-term patterns. Overall, the results show that well-planned recalibration, combined with appropriate class balancing, can help keep crash prediction systems reliable over time.

Appendix A. Additional Tables

Table A.6: Prediction results for XGBoost, AdaBoost, Random Forest, and NNet without recalibration (static model), **with** and **without SMOTE balancing** and using the Youden criterion for threshold selection.

Iter.	Training	Prediction	Model	Balance	AUC	PR-AUC	F1	Brier	Sensitivity	Specificity	Error Rate
1	2018	2019	AdaBoost	none	0.7540	0.0094	0.0144	0.0025	0.4921	0.8319	0.1690
2	2018	2020	AdaBoost	none	0.7364	0.0100	0.0184	0.0026	0.5318	0.8542	0.1466
3	2018	2021	AdaBoost	none	0.7659	0.0113	0.0154	0.0027	0.6145	0.7889	0.2116
4	2018	2022	AdaBoost	none	0.7503	0.0112	0.0127	0.0022	0.5485	0.8136	0.1869
5	2018	2023	AdaBoost	none	0.8069	0.0142	0.0156	0.0023	0.6386	0.8169	0.1835
6	2018	2024	AdaBoost	none	0.7525	0.0111	0.0164	0.0026	0.5649	0.8268	0.1738
1	2018	2019	AdaBoost	smote	0.7460	0.0082	0.0117	0.0223	0.7222	0.6951	0.3048
2	2018	2020	AdaBoost	smote	0.7207	0.0076	0.0125	0.0217	0.6185	0.7478	0.2525
3	2018	2021	AdaBoost	smote	0.7489	0.0092	0.0119	0.0232	0.7430	0.6658	0.3340
4	2018	2022	AdaBoost	smote	0.7445	0.0075	0.0098	0.0224	0.7184	0.6808	0.3192
5	2018	2023	AdaBoost	smote	0.8086	0.0110	0.0122	0.0224	0.8253	0.6946	0.3051
6	2018	2024	AdaBoost	smote	0.7265	0.0090	0.0121	0.0220	0.6883	0.7130	0.2871
1	2018	2019	NNet	none	0.5058	0.0026	0.0038	0.0027	0.0794	0.8981	0.1040
2	2018	2020	NNet	none	0.5538	0.0045	0.0108	0.0027	0.1503	0.9311	0.0710
3	2018	2021	NNet	none	0.5263	0.0033	0.0079	0.0028	0.0803	0.9476	0.0547
4	2018	2022	NNet	none	0.4602	0.0034	0.0036	0.0023	0.0680	0.9200	0.0819
5	2018	2023	NNet	none	0.4972	0.0029	0.0050	0.0024	0.0964	0.9138	0.0881
6	2018	2024	NNet	none	0.4954	0.0027	0.0062	0.0027	0.1234	0.9001	0.1019
1	2018	2019	NNet	smote	0.7515	0.0125	0.0147	0.0025	0.5873	0.8036	0.1970
2	2018	2020	NNet	smote	0.7045	0.0134	0.0138	0.0026	0.5260	0.8067	0.1940
3	2018	2021	NNet	smote	0.7670	0.0187	0.0141	0.0027	0.6466	0.7560	0.2443
4	2018	2022	NNet	smote	0.7454	0.0198	0.0111	0.0022	0.5971	0.7684	0.2320
5	2018	2023	NNet	smote	0.8060	0.0185	0.0146	0.0023	0.7108	0.7812	0.2190
6	2018	2024	NNet	smote	0.7415	0.0226	0.0159	0.0025	0.6299	0.8010	0.1994
1	2018	2019	Random Forest	none	0.7733	0.0155	0.0123	0.0025	0.6984	0.7180	0.2821
2	2018	2020	Random Forest	none	0.7608	0.0125	0.0139	0.0026	0.6821	0.7493	0.2508
3	2018	2021	Random Forest	none	0.7879	0.0136	0.0125	0.0027	0.7430	0.6838	0.3161
4	2018	2022	Random Forest	none	0.7750	0.0122	0.0110	0.0022	0.7282	0.7146	0.2853
5	2018	2023	Random Forest	none	0.8371	0.0205	0.0131	0.0023	0.8133	0.7197	0.2801
6	2018	2024	Random Forest	none	0.7650	0.0219	0.0130	0.0025	0.7013	0.7269	0.2731
1	2018	2019	Random Forest	smote	0.7774	0.0148	0.0116	0.0025	0.7698	0.6716	0.3282
2	2018	2020	Random Forest	smote	0.7496	0.0115	0.0130	0.0026	0.6763	0.7351	0.2650
3	2018	2021	Random Forest	smote	0.7886	0.0236	0.0120	0.0027	0.7831	0.6525	0.3471
4	2018	2022	Random Forest	smote	0.7826	0.0202	0.0097	0.0022	0.7573	0.6615	0.3383
5	2018	2023	Random Forest	smote	0.8467	0.0233	0.0116	0.0023	0.8494	0.6684	0.3312
6	2018	2024	Random Forest	smote	0.7803	0.0284	0.0118	0.0026	0.7273	0.6876	0.3123
1	2018	2019	XGBoost	none	0.7811	0.0144	0.0095	0.0025	0.8016	0.5822	0.4173
2	2018	2020	XGBoost	none	0.7322	0.0137	0.0106	0.0026	0.7110	0.6576	0.3423
3	2018	2021	XGBoost	none	0.7870	0.0237	0.0103	0.0027	0.8153	0.5784	0.4210
4	2018	2022	XGBoost	none	0.7733	0.0240	0.0081	0.0022	0.7816	0.5826	0.4170
5	2018	2023	XGBoost	none	0.8378	0.0266	0.0096	0.0023	0.8614	0.5929	0.4065
6	2018	2024	XGBoost	none	0.7599	0.0270	0.0097	0.0025	0.7532	0.6053	0.3943
1	2018	2019	XGBoost	smote	0.7780	0.0139	0.0102	0.0025	0.7937	0.6141	0.3854
2	2018	2020	XGBoost	smote	0.7407	0.0138	0.0114	0.0026	0.6821	0.6953	0.3048
3	2018	2021	XGBoost	smote	0.7858	0.0245	0.0109	0.0027	0.7831	0.6164	0.3832
4	2018	2022	XGBoost	smote	0.7681	0.0203	0.0084	0.0022	0.7524	0.6112	0.3885
5	2018	2023	XGBoost	smote	0.8345	0.0264	0.0102	0.0023	0.8494	0.6245	0.3750
6	2018	2024	XGBoost	smote	0.7560	0.0289	0.0101	0.0025	0.7273	0.6348	0.3649

Table A.7: Prediction results for XGBoost, AdaBoost, Random Forest, and NNet with period-aligned retraining, [with and without SMOTE balancing](#).

Iter.	Training	Prediction	Model	Balance	AUC	PR-AUC	F1	Brier	Sensitivity	Specificity	Error Rate
1	2018	2019	AdaBoost	none	0.7540	0.0094	0.0144	0.0025	0.4921	0.8319	0.1690
1	2018	2019	AdaBoost	smote	0.7460	0.0082	0.0117	0.0223	0.7222	0.6951	0.3048
2	2019	2020	AdaBoost	none	0.6917	0.0058	0.0113	0.0227	0.4566	0.7951	0.2058
2	2019	2020	AdaBoost	smote	0.7357	0.0092	0.0112	0.0224	0.6936	0.6823	0.3177
3	2020	2021	AdaBoost	none	0.7341	0.0099	0.0065	0.0314	0.9598	0.2120	0.7859
3	2020	2021	AdaBoost	smote	0.7432	0.0109	0.0156	0.0027	0.5060	0.8288	0.1720
4	2021	2022	AdaBoost	none	0.7639	0.0163	0.0104	0.0022	0.6699	0.7204	0.2797
4	2021	2022	AdaBoost	smote	0.7329	0.0125	0.0143	0.0022	0.4272	0.8720	0.1290
5	2022	2023	AdaBoost	none	0.7985	0.0166	0.0207	0.1193	0.5422	0.8839	0.1169
5	2022	2023	AdaBoost	smote	0.7977	0.0210	0.0171	0.0908	0.6446	0.8316	0.1688
6	2023	2024	AdaBoost	none	0.7551	0.0313	0.0190	0.0026	0.5519	0.8549	0.1459
6	2023	2024	AdaBoost	smote	0.7344	0.0149	0.0175	0.0025	0.5455	0.8435	0.1573
1	2018	2019	NNet	none	0.5058	0.0026	0.0038	0.0027	0.0794	0.8981	0.1040
1	2018	2019	NNet	smote	0.7515	0.0125	0.0147	0.0025	0.5873	0.8036	0.1970
2	2019	2020	NNet	none	0.4868	0.0026	0.0052	0.0035	0.1908	0.8146	0.1870
2	2019	2020	NNet	smote	0.6863	0.0078	0.0105	0.0026	0.5376	0.7396	0.2609
3	2020	2021	NNet	none	0.3907	0.0023	0.0041	0.0066	0.4618	0.3878	0.6120
3	2020	2021	NNet	smote	0.6857	0.0093	0.0138	0.0027	0.5462	0.7895	0.2112
4	2021	2022	NNet	none	0.4722	0.0022	0.0044	0.0148	1.0000	0.0000	0.9978
4	2021	2022	NNet	smote	0.7465	0.0254	0.0090	0.0022	0.7427	0.6419	0.3578
5	2022	2023	NNet	none	0.4294	0.0019	0.0000	0.0049	0.0000	1.0000	0.0023
5	2022	2023	NNet	smote	0.7669	0.0197	0.0113	0.0023	0.6867	0.7263	0.2738
6	2023	2024	NNet	none	0.4804	0.0025	0.0048	0.0082	0.3312	0.6459	0.3549
6	2023	2024	NNet	smote	0.7478	0.0253	0.0199	0.0025	0.5844	0.8534	0.1473
1	2018	2019	Random Forest	none	0.7733	0.0155	0.0123	0.0025	0.6984	0.7180	0.2821
1	2018	2019	Random Forest	smote	0.7774	0.0148	0.0116	0.0025	0.7698	0.6716	0.3282
2	2019	2020	Random Forest	none	0.7394	0.0128	0.0056	0.0026	0.9249	0.1431	0.8549
2	2019	2020	Random Forest	smote	0.7291	0.0124	0.0102	0.0026	0.7283	0.6354	0.3644
3	2020	2021	Random Forest	none	0.7467	0.0124	0.0110	0.0034	0.7068	0.6560	0.3439
3	2020	2021	Random Forest	smote	0.7459	0.0124	0.0095	0.0027	0.7349	0.5885	0.4111
4	2021	2022	Random Forest	none	0.7680	0.0173	0.0137	0.0022	0.6165	0.8058	0.1946
4	2021	2022	Random Forest	smote	0.7585	0.0178	0.0113	0.0022	0.6019	0.7703	0.2301
5	2022	2023	Random Forest	none	0.8410	0.0220	0.0173	0.0023	0.7530	0.8045	0.1956
5	2022	2023	Random Forest	smote	0.8485	0.0292	0.0162	0.0023	0.7590	0.7896	0.2104
6	2023	2024	Random Forest	none	0.8138	0.0325	0.0205	0.0025	0.6429	0.8433	0.1572
6	2023	2024	Random Forest	smote	0.8173	0.0339	0.0174	0.0026	0.6948	0.7992	0.2011
1	2018	2019	XGBoost	none	0.7811	0.0144	0.0095	0.0025	0.8016	0.5822	0.4173
1	2018	2019	XGBoost	smote	0.7780	0.0139	0.0102	0.0025	0.7937	0.6141	0.3854
2	2019	2020	XGBoost	none	0.7493	0.0105	0.0150	0.0026	0.6243	0.7880	0.2125
2	2019	2020	XGBoost	smote	0.7620	0.0107	0.0154	0.0026	0.6185	0.7960	0.2044
3	2020	2021	XGBoost	none	0.7680	0.0185	0.0127	0.0027	0.7189	0.6985	0.3015
3	2020	2021	XGBoost	smote	0.7511	0.0161	0.0143	0.0027	0.6104	0.7734	0.2270
4	2021	2022	XGBoost	none	0.7629	0.0235	0.0112	0.0022	0.6699	0.7400	0.2601
4	2021	2022	XGBoost	smote	0.7350	0.0168	0.0083	0.0022	0.7379	0.6158	0.3839
5	2022	2023	XGBoost	none	0.8387	0.0294	0.0161	0.0023	0.7711	0.7852	0.2149
5	2022	2023	XGBoost	smote	0.8376	0.0302	0.0141	0.0023	0.7952	0.7454	0.2545
6	2023	2024	XGBoost	none	0.7848	0.0247	0.0168	0.0026	0.6429	0.8073	0.1931
6	2023	2024	XGBoost	smote	0.7888	0.0216	0.0188	0.0026	0.6104	0.8377	0.1629

Table A.8: Prediction results for XGBoost, AdaBoost, Random Forest, and NNet with cumulative retraining, [with and without SMOTE balancing](#).

Iter.	Training	Prediction	Model	Balance	AUC	PR-AUC	F1	Brier	Sensitivity	Specificity	Error Rate
1	[$\leq$ 2018]	2019	AdaBoost	none	0.7540	0.0094	0.0144	0.0025	0.4921	0.8319	0.1690
2	[$\leq$ 2019]	2020	AdaBoost	none	0.7392	0.0118	0.0216	0.0026	0.4855	0.8875	0.1135
3	[$\leq$ 2020]	2021	AdaBoost	none	0.7849	0.0243	0.0175	0.0027	0.5984	0.8193	0.1813
4	[$\leq$ 2021]	2022	AdaBoost	none	0.7696	0.0218	0.0299	0.0022	0.3204	0.9559	0.0455
5	[$\leq$ 2022]	2023	AdaBoost	none	0.8395	0.0279	0.0195	0.0023	0.7349	0.8316	0.1686
6	[$\leq$ 2023]	2024	AdaBoost	none	0.7800	0.0166	0.0248	0.0025	0.5000	0.9005	0.1005
1	[$\leq$ 2018]	2019	AdaBoost	smote	0.7460	0.0082	0.0117	0.0223	0.7222	0.6951	0.3048
2	[$\leq$ 2019]	2020	AdaBoost	smote	0.7270	0.0113	0.0214	0.0026	0.4335	0.8987	0.1025
3	[$\leq$ 2020]	2021	AdaBoost	smote	0.7682	0.0142	0.0159	0.0306	0.6145	0.7948	0.2057
4	[$\leq$ 2021]	2022	AdaBoost	smote	0.7468	0.0177	0.0190	0.0022	0.4563	0.8978	0.1031
5	[$\leq$ 2022]	2023	AdaBoost	smote	0.8348	0.0243	0.0171	0.0208	0.7651	0.7994	0.2007
6	[$\leq$ 2023]	2024	AdaBoost	smote	0.7733	0.0261	0.0137	0.0216	0.6558	0.7582	0.2421
1	[$\leq$ 2018]	2019	NNet	none	0.5058	0.0026	0.0038	0.0027	0.0794	0.8981	0.1040
2	[$\leq$ 2019]	2020	NNet	none	0.5561	0.0039	0.0066	0.0027	0.5780	0.5515	0.4485
3	[$\leq$ 2020]	2021	NNet	none	0.5285	0.0040	0.0066	0.0028	0.3815	0.6909	0.3099
4	[$\leq$ 2021]	2022	NNet	none	0.4609	0.0029	0.0039	0.0026	0.5777	0.3568	0.6427
5	[$\leq$ 2022]	2023	NNet	none	0.5250	0.0036	0.0055	0.0025	0.5000	0.5889	0.4113
6	[$\leq$ 2023]	2024	NNet	none	0.5251	0.0029	0.0064	0.0027	0.3312	0.7385	0.2625
1	[$\leq$ 2018]	2019	NNet	smote	0.7515	0.0125	0.0147	0.0025	0.5873	0.8036	0.1970
2	[$\leq$ 2019]	2020	NNet	smote	0.7283	0.0139	0.0125	0.0026	0.6590	0.7322	0.2680
3	[$\leq$ 2020]	2021	NNet	smote	0.7759	0.0219	0.0120	0.0027	0.7470	0.6686	0.3311
4	[$\leq$ 2021]	2022	NNet	smote	0.7671	0.0249	0.0115	0.0022	0.6796	0.7442	0.2560
5	[$\leq$ 2022]	2023	NNet	smote	0.8311	0.0251	0.0123	0.0023	0.8193	0.6989	0.3008
6	[$\leq$ 2023]	2024	NNet	smote	0.7848	0.0258	0.0181	0.0025	0.6558	0.8182	0.1822
1	[$\leq$ 2018]	2019	Random Forest	none	0.7733	0.0155	0.0123	0.0025	0.6984	0.7180	0.2821
2	[$\leq$ 2019]	2020	Random Forest	none	0.7740	0.0146	0.0169	0.0026	0.5896	0.8229	0.1778
3	[$\leq$ 2020]	2021	Random Forest	none	0.7958	0.0174	0.0156	0.0027	0.6506	0.7796	0.2208
4	[$\leq$ 2021]	2022	Random Forest	none	0.7860	0.0205	0.0167	0.0022	0.6311	0.8377	0.1628
5	[$\leq$ 2022]	2023	Random Forest	none	0.8393	0.0328	0.0182	0.0023	0.7169	0.8237	0.1766
6	[$\leq$ 2023]	2024	Random Forest	none	0.8044	0.0317	0.0202	0.0025	0.6299	0.8439	0.1567
1	[$\leq$ 2018]	2019	Random Forest	smote	0.7774	0.0148	0.0116	0.0025	0.7698	0.6716	0.3282
2	[$\leq$ 2019]	2020	Random Forest	smote	0.7577	0.0127	0.0155	0.0026	0.6127	0.7995	0.2010
3	[$\leq$ 2020]	2021	Random Forest	smote	0.7879	0.0235	0.0159	0.0027	0.6867	0.7712	0.2291
4	[$\leq$ 2021]	2022	Random Forest	smote	0.7829	0.0231	0.0120	0.0022	0.7282	0.7378	0.2622
5	[$\leq$ 2022]	2023	Random Forest	smote	0.8602	0.0355	0.0179	0.0023	0.7651	0.8090	0.1911
6	[$\leq$ 2023]	2024	Random Forest	smote	0.8140	0.0536	0.0165	0.0026	0.6948	0.7884	0.2118
1	[$\leq$ 2018]	2019	XGBoost	none	0.7811	0.0144	0.0095	0.0025	0.8016	0.5822	0.4173
2	[$\leq$ 2019]	2020	XGBoost	none	0.7527	0.0162	0.0161	0.0026	0.6185	0.8056	0.1949
3	[$\leq$ 2020]	2021	XGBoost	none	0.7966	0.0310	0.0159	0.0027	0.6386	0.7872	0.2132
4	[$\leq$ 2021]	2022	XGBoost	none	0.7945	0.0294	0.0204	0.0022	0.5680	0.8812	0.1195
5	[$\leq$ 2022]	2023	XGBoost	none	0.8658	0.0368	0.0178	0.0023	0.8012	0.7987	0.2012
6	[$\leq$ 2023]	2024	XGBoost	none	0.7993	0.0336	0.0205	0.0026	0.5909	0.8563	0.1443
1	[$\leq$ 2018]	2019	XGBoost	smote	0.7780	0.0139	0.0102	0.0025	0.7937	0.6141	0.3854
2	[$\leq$ 2019]	2020	XGBoost	smote	0.7446	0.0163	0.0155	0.0026	0.6127	0.7998	0.2007
3	[$\leq$ 2020]	2021	XGBoost	smote	0.7883	0.0205	0.0144	0.0027	0.6988	0.7418	0.2584
4	[$\leq$ 2021]	2022	XGBoost	smote	0.7899	0.0199	0.0114	0.0022	0.7330	0.7222	0.2778
5	[$\leq$ 2022]	2023	XGBoost	smote	0.8560	0.0369	0.0154	0.0023	0.8434	0.7529	0.2469
6	[$\leq$ 2023]	2024	XGBoost	smote	0.8125	0.0523	0.0195	0.0026	0.6234	0.8401	0.1605

Table A.9: Adaptive Random Forest (ARF) results by voting strategy and prediction year. ARF processes data as an online stream without batch retraining.

ARF Variant	Prediction Year	AUC	F1	Brier	Sensitivity	Specificity	Error Rate
ARFfast	2019	0.4397	0.0045	0.0025	0.3413	0.6248	0.3759
ARFfast	2020	0.4692	0.0000	0.0026	0.0000	1.0000	0.0026
ARFfast	2021	0.4719	0.0000	0.0027	0.0000	1.0000	0.0027
ARFfast	2022	0.4872	0.0000	0.0022	0.0000	1.0000	0.0022
ARFfast	2023	0.4682	0.0000	0.0023	0.0000	1.0000	0.0023
ARFfast	2024	0.4764	0.0000	0.0026	0.0000	1.0000	0.0026
ARFmaj	2019	0.4374	0.0044	0.0025	0.3413	0.6117	0.3890
ARFmaj	2020	0.4695	0.0031	0.0026	0.0058	0.9929	0.0096
ARFmaj	2021	0.4733	0.0000	0.0027	0.0000	1.0000	0.0027
ARFmaj	2022	0.4893	0.0000	0.0022	0.0000	1.0000	0.0022
ARFmaj	2023	0.4781	0.0000	0.0023	0.0000	1.0000	0.0023
ARFmaj	2024	0.4790	0.0000	0.0026	0.0000	1.0000	0.0026
ARFmoderate	2019	0.4374	0.0044	0.0025	0.3413	0.6117	0.3890
ARFmoderate	2020	0.4695	0.0031	0.0026	0.0058	0.9929	0.0096
ARFmoderate	2021	0.4733	0.0000	0.0027	0.0000	1.0000	0.0027
ARFmoderate	2022	0.4893	0.0000	0.0022	0.0000	1.0000	0.0022
ARFmoderate	2023	0.4781	0.0000	0.0023	0.0000	1.0000	0.0023
ARFmoderate	2024	0.4790	0.0000	0.0026	0.0000	1.0000	0.0026

Table A.10: Evaluation metrics by model and ADWIN drift detection iteration, with and without SMOTE balancing.

Drift	Drift Date	W	W0	W1	Rem. Periods	Model	Balance	AUC	F1	Brier	Sensitivity	Specificity	Error Rate
1	2019-06-11	45000	32	44968	391503	XGBoost	none	0.7761	0.0097	0.0025	0.7807	0.5955	0.4040
2	2019-06-11	45000	383	44617	391471	XGBoost	none	–	0.0000	0.0000	–	0.6250	0.3750
3	2019-06-12	45000	44609	391	391088	XGBoost	none	–	0.0000	0.0000	–	0.4073	0.5927
4	2020-09-15	45000	1085	43915	346479	XGBoost	none	0.7637	0.0123	0.0027	0.7083	0.6941	0.3059
5	2020-09-19	45000	5297	39703	345394	XGBoost	none	0.1255	0.0000	0.0009	0.0000	0.5830	0.4175
6	2020-10-08	45000	44960	40	340097	XGBoost	none	0.7102	0.0067	0.0026	0.7143	0.4378	0.5614
7	2021-04-16	45000	44960	40	295137	XGBoost	none	0.7719	0.0069	0.0023	0.8725	0.4310	0.5680
8	2021-10-11	45000	32	44968	250177	XGBoost	none	0.7520	0.0089	0.0030	0.8394	0.4315	0.5673
9	2021-10-11	45000	44960	40	250145	XGBoost	none	–	0.0000	0.0000	–	0.9688	0.0312
10	2022-03-25	45000	44258	742	205185	XGBoost	none	0.7281	0.0113	0.0023	0.5294	0.7898	0.2108
11	2022-09-16	45000	43907	1093	160927	XGBoost	none	0.7391	0.0128	0.0024	0.5833	0.7806	0.2199
12	2023-03-16	45000	383	44617	117020	XGBoost	none	0.7658	0.0130	0.0021	0.6087	0.8068	0.1936
13	2023-03-17	45000	44609	391	116637	XGBoost	none	0.5969	0.0112	0.0026	1.0000	0.5393	0.4595
14	2023-11-20	45000	383	44617	72028	XGBoost	none	0.8227	0.0141	0.0024	0.7664	0.7424	0.2575
15	2023-11-21	45000	383	44617	71645	XGBoost	none	0.9948	0.0211	0.0026	1.0000	0.7565	0.2428
16	2023-11-23	45000	43556	1444	71262	XGBoost	none	0.8848	0.0187	0.0026	1.0000	0.7251	0.2742
17	2024-05-15	45000	383	44617	27706	XGBoost	none	0.8184	0.0107	0.0019	0.8193	0.7121	0.2877
18	2024-05-17	45000	44258	742	27323	XGBoost	none	0.5879	0.0163	0.0052	0.5000	0.6850	0.3159
19	2024-09-30	28065	–	–	0	XGBoost	none	0.7331	0.0178	0.0032	0.5843	0.7905	0.2102
1	2019-06-11	45000	383	44617	391503	XGBoost	smote	0.7735	0.0105	0.0025	0.7807	0.6252	0.3744
2	2019-06-12	45000	44960	40	391120	XGBoost	smote	–	0.0000	0.0032	–	0.3342	0.6658
3	2020-09-17	45000	32	44968	346160	XGBoost	smote	0.7594	0.0119	0.0046	0.6917	0.6930	0.3070
4	2020-09-17	45000	32	44968	346128	XGBoost	smote	–	0.0000	0.0000	–	1.0000	0.0000
5	2020-09-17	45000	32	44968	346096	XGBoost	smote	–	0.0000	0.0000	–	0.9688	0.0312
6	2020-09-17	45000	32	44968	346064	XGBoost	smote	0.7240	0.0105	0.0033	0.6485	0.6738	0.3261
7	2020-09-19	45000	32	44968	346032	XGBoost	smote	–	0.0000	0.0000	–	0.7812	0.2188
8	2020-09-19	45000	1085	43915	346000	XGBoost	smote	–	0.0000	0.0140	–	0.4688	0.5312
9	2020-09-23	45000	44960	40	344915	XGBoost	smote	0.7158	0.0115	0.0028	0.5475	0.7148	0.2855
10	2021-02-24	45000	37366	7634	299955	XGBoost	smote	0.7184	0.0097	0.0024	0.8750	0.4792	0.5198
11	2021-04-16	45000	44960	40	262589	XGBoost	smote	0.7747	0.0098	0.0024	0.8235	0.5007	0.4984
12	2021-10-12	45000	44960	40	217629	XGBoost	smote	0.7181	0.0082	0.0023	0.8285	0.4209	0.5788
13	2022-03-25	45000	44960	40	172669	XGBoost	smote	0.7715	0.0095	0.0023	0.7500	0.5246	0.4748
14	2022-03-25	45000	32	44968	172637	XGBoost	smote	–	0.0000	0.0000	–	0.9688	0.0312
15	2022-03-25	45000	10576	34424	172605	XGBoost	smote	0.6994	0.0110	0.0022	0.5500	0.7280	0.2713
16	2023-03-15	45000	44258	742	127920	XGBoost	smote	0.7546	0.0140	0.0021	0.6296	0.7840	0.2165
17	2023-11-15	45000	32	44968	83662	XGBoost	smote	0.9004	0.0202	0.0020	0.7381	0.8385	0.1613
18	2023-11-15	45000	43556	1444	83630	XGBoost	smote	–	0.0000	0.0000	–	0.7812	0.2188
19	2023-11-16	45000	44258	742	40074	XGBoost	smote	0.7480	0.0241	0.0023	0.5229	0.9007	0.1005
20	2024-04-13	45000	44960	40	31893	XGBoost	smote	0.8213	0.0172	0.0020	0.8443	0.7502	0.2496
21	2024-04-13	45000	32	44968	31861	XGBoost	smote	–	0.0000	0.0000	–	0.9062	0.0938
22	2024-04-13	45000	44258	742	31829	XGBoost	smote	0.7268	0.0170	0.0021	0.6000	0.8499	0.1507
23	2024-08-25	45000	44258	742	7571	XGBoost	smote	0.7017	0.0200	0.0022	0.5000	0.8755	0.1254
24	2024-09-24	45000	742	44258	6829	XGBoost	smote	0.4975	0.0058	0.0021	0.3333	0.7963	0.2042
25	2024-09-27	45000	1085	43915	6087	XGBoost	smote	–	0.0000	0.0000	–	0.8125	0.1875
26	2024-09-27	45000	17231	27769	5002	XGBoost	smote	–	0.0000	0.0000	–	0.7500	0.2500
27	2024-09-27	45000	29165	15835	1061	XGBoost	smote	0.7576	0.0092	0.0033	0.8846	0.3736	0.6248
28	2024-09-30	16896	–	–	0	XGBoost	smote	0.2181	0.0000	0.0019	0.0000	0.6062	0.3949
1	2019-06-11	45000	17933	27067	391503	AdaBoost	none	0.7524	0.0154	0.0025	0.5000	0.8392	0.1616
2	2020-02-28	45000	44960	40	373570	AdaBoost	none	0.6967	0.0078	0.0022	0.7000	0.6033	0.3965
3	2020-11-28	45000	32	44968	328610	AdaBoost	none	0.6886	0.0099	0.0027	0.5537	0.7016	0.2988
4	2020-11-28	45000	32	44968	328578	AdaBoost	none	0.7097	0.1111	0.0466	1.0000	0.4839	0.5000

Continues on next page

Drift	Drift Date	W	W0	W1	Rem. Periods	Model	Balance	AUC	F1	Brier	Sensitivity	Specificity	Error Rate
5	2020-11-28	45000	42503	2497	328546	AdaBoost	none	—	0.0000	0.0170	—	0.9688	0.0312
6	2021-06-06	45000	44960	40	286043	AdaBoost	none	0.7259	0.0083	0.0022	0.8490	0.3603	0.6386
7	2021-10-13	45000	17231	27769	241083	AdaBoost	none	0.6171	0.0042	0.0022	0.8382	0.2509	0.7488
8	2021-10-16	45000	44258	742	223852	AdaBoost	none	0.7301	0.0095	0.0022	0.6813	0.5529	0.4469
9	2021-10-16	45000	44960	40	179594	AdaBoost	none	0.7555	0.0105	0.0022	0.6324	0.6253	0.3739
10	2022-03-25	45000	43907	1093	134634	AdaBoost	none	0.7810	0.0168	0.0022	0.5172	0.8911	0.1094
11	2022-09-09	45000	39100	5900	90695	AdaBoost	none	0.7667	0.0112	0.0024	0.6577	0.7570	0.2428
12	2022-10-09	45000	44960	40	51595	AdaBoost	none	0.8227	0.0135	0.0023	0.6667	0.8111	0.1889
13	2023-07-31	45000	44960	40	6635	AdaBoost	none	0.8051	0.0125	0.0021	0.7619	0.7459	0.2535
14	2023-08-03	45000	32	44968	6603	AdaBoost	none	—	0.0000	0.0000	—	0.8125	0.1875
15	2023-08-03	45000	32	44968	6571	AdaBoost	none	—	0.0000	0.0000	—	0.9062	0.0938
16	2023-08-03	45000	17231	27769	6539	AdaBoost	none	—	0.0000	0.0174	—	0.6250	0.3750
17	2023-08-03	45000	1085	43915	647	AdaBoost	none	—	0.0000	0.0000	—	0.9062	0.0938
18	2023-08-03	45000	383	44617	615	AdaBoost	none	—	0.0000	0.0000	—	0.7812	0.2188
19	2023-08-04	45000	17231	27769	232	AdaBoost	none	—	0.0000	0.0000	—	0.5312	0.4688
20	2023-08-04	45000	44960	40	200	AdaBoost	none	—	0.0000	0.0000	—	0.4688	0.5312
21	2023-08-04	45000	32	44968	168	AdaBoost	none	—	0.0000	0.0000	—	0.9062	0.0938
22	2023-08-04	45000	44258	742	136	AdaBoost	none	—	0.0000	0.0000	—	0.8750	0.1250
23	2023-08-04	45000	44960	40	104	AdaBoost	none	—	0.0000	0.0000	—	0.9062	0.0938
24	2024-04-28	45000	44609	391	32747	AdaBoost	none	—	0.0000	0.0159	—	1.0000	0.0000
25	2024-09-30	33138	—	—	0	AdaBoost	none	0.7236	0.0219	0.0221	0.4216	0.8840	0.1174
1	2019-06-11	45000	32	44968	391503	AdaBoost	smote	0.7465	0.0122	0.0221	0.7193	0.7037	0.2962
2	2019-06-11	45000	383	44617	391471	AdaBoost	smote	—	0.0000	0.0000	—	0.2188	0.7812
3	2019-06-12	45000	44960	40	391088	AdaBoost	smote	—	0.0000	0.0288	—	0.4595	0.5405
4	2020-09-17	45000	32	44968	346128	AdaBoost	smote	0.6833	0.0116	0.0254	0.4750	0.7851	0.2157
5	2020-09-17	45000	32	44968	346096	AdaBoost	smote	—	0.0000	0.0000	—	1.0000	0.0000
6	2020-09-17	45000	11966	33034	346064	AdaBoost	smote	—	0.0000	0.0000	—	0.2500	0.7500
7	2020-10-04	45000	17582	27418	334098	AdaBoost	smote	0.7346	0.0118	0.0027	0.6739	0.6710	0.3289
8	2021-06-22	45000	5911	39089	316516	AdaBoost	smote	0.7583	0.0141	0.0024	0.6154	0.7818	0.2174
9	2021-06-22	45000	32	44968	310605	AdaBoost	smote	—	0.0000	0.0000	—	0.9688	0.0312
10	2021-06-22	45000	32	44968	310573	AdaBoost	smote	—	0.0000	0.0000	—	0.7188	0.2812
11	2021-06-23	45000	742	44258	310541	AdaBoost	smote	0.7320	0.0126	0.0024	0.6154	0.7624	0.2368
12	2021-06-23	45000	42311	2689	309799	AdaBoost	smote	—	0.0000	0.0000	—	0.8438	0.1562
13	2023-04-06	139731	139680	51	111060	AdaBoost	smote	0.7121	0.0182	0.0023	0.2812	0.9328	0.0686
14	2024-09-30	111111	—	—	0	AdaBoost	smote	0.7204	0.0228	0.0024	0.2926	0.9407	0.0608
1	2019-06-11	45085	44736	349	391418	NNet	none	0.4954	0.0030	0.0028	0.0614	0.9000	0.1021
2	2020-09-14	45000	383	44617	346767	NNet	none	0.5966	0.0124	0.0028	0.1849	0.9232	0.0787
3	2020-09-16	45000	11966	33034	346384	NNet	none	0.2225	0.0000	0.0029	0.0000	0.9293	0.0731
4	2020-10-29	45000	43556	1444	334418	NNet	none	0.5116	0.0062	0.0057	0.3226	0.7326	0.2685
5	2021-05-01	45000	44609	391	290862	NNet	none	0.3690	0.0026	0.0053	0.1633	0.7207	0.2806
6	2021-10-11	45000	17231	27769	246253	NNet	none	0.2879	0.0047	0.0038	0.3862	0.4938	0.5050
7	2022-03-21	45000	17231	27769	229022	NNet	none	0.4845	0.0062	0.0030	0.3333	0.6855	0.3152
8	2022-09-08	45000	32	44968	211791	NNet	none	0.3176	0.0011	0.0022	1.0000	0.0115	0.9885
9	2022-09-08	45000	115	44885	211759	NNet	none	0.4228	0.0040	0.0023	0.7273	0.3512	0.6480
10	2022-09-08	45000	103	44897	211644	NNet	none	0.5983	0.0051	0.0023	0.7000	0.4985	0.5010
11	2022-09-08	45000	103	44897	211541	NNet	none	0.5280	0.0026	0.0024	0.8000	0.2915	0.7070
12	2022-10-09	45000	27366	17634	211438	NNet	none	0.4244	0.0034	0.0048	0.6667	0.3604	0.6394
13	2023-04-26	45000	5911	39089	184072	NNet	none	0.5776	0.0156	0.0023	0.3000	0.9069	0.0938
14	2023-04-26	45000	1085	43915	178161	NNet	none	0.5241	0.0071	0.0025	0.4286	0.6690	0.3316
15	2023-04-26	45000	17231	27769	177076	NNet	none	0.5102	0.0064	0.0068	0.5500	0.4686	0.5315
16	2023-08-22	45000	44258	742	159845	NNet	none	0.2706	0.0048	0.0069	0.3333	0.5942	0.4069
17	2024-09-24	45000	44960	40	15613	NNet	none	0.6121	0.0116	0.0028	0.5410	0.7900	0.2107
18	2024-09-25	45000	44960	40	1573	NNet	none	0.4800	0.0064	0.0145	0.2985	0.7006	0.3007

Continues on next page

Drift	Drift Date	W	W0	W1	Rem. Periods	Model	Balance	AUC	F1	Brier	Sensitivity	Specificity	Error Rate
19	2024-09-30	1613	—	—	0	NNet	none	0.5344	0.0082	0.0150	0.3333	0.6950	0.3064
1	2019-06-11	45000	17582	27418	391503	NNet	smote	0.7532	0.0152	0.0025	0.5877	0.8078	0.1928
2	2020-02-27	45000	27410	17590	373921	NNet	smote	0.5518	0.0044	0.0123	0.5135	0.5146	0.4854
3	2020-09-15	45000	383	44617	346511	NNet	smote	0.6469	0.0240	0.0082	0.2048	0.9517	0.0505
4	2020-09-17	45000	32	44968	346128	NNet	smote	—	0.0000	0.0000	—	0.3995	0.6005
5	2020-09-17	45000	17231	27769	346096	NNet	smote	—	0.0000	0.0000	—	1.0000	0.0000
6	2020-09-17	45000	17231	27769	328865	NNet	smote	0.7029	0.0154	0.0027	0.4900	0.8247	0.1759
7	2021-05-13	45000	44375	625	311634	NNet	smote	0.7984	0.0126	0.0023	0.6750	0.7504	0.2497
8	2021-05-14	45000	44258	742	267259	NNet	smote	0.6535	0.0102	0.0024	0.5957	0.7213	0.2790
9	2021-05-14	45000	43556	1444	223001	NNet	smote	0.7342	0.0089	0.0025	0.7285	0.6428	0.3573
10	2022-03-23	45000	44960	40	179445	NNet	smote	0.7465	0.0107	0.0022	0.6742	0.6972	0.3030
11	2022-03-23	45000	44258	742	134485	NNet	smote	0.7471	0.0111	0.0022	0.5641	0.7588	0.2417
12	2022-11-17	45000	44960	40	90227	NNet	smote	0.7845	0.0139	0.0022	0.5333	0.8495	0.1498
13	2023-04-17	45000	44960	40	45267	NNet	smote	0.7933	0.0168	0.0023	0.5952	0.8526	0.1477
14	2024-03-22	45000	41558	3442	307	NNet	smote	0.7134	0.0211	0.0027	0.5333	0.8658	0.1345
15	2024-03-22	45000	44609	391	275	NNet	smote	—	0.0000	0.0000	—	0.7812	0.2188
16	2024-04-07	45000	44960	40	38776	NNet	smote	0.7990	0.0251	0.0021	0.5591	0.9100	0.0908
17	2024-09-30	38816	—	—	0	NNet	smote	0.7298	0.0249	0.0028	0.4091	0.9104	0.0910
1	2019-06-11	45000	32	44968	391503	Random Forest	none	0.7611	0.0124	0.0025	0.6754	0.7278	0.2724
2	2019-06-11	45000	17933	27067	391471	Random Forest	none	—	0.0000	0.0000	—	0.5938	0.4062
3	2020-02-28	45000	37238	7762	373538	Random Forest	none	0.7578	0.0119	0.0022	0.5500	0.7962	0.2043
4	2020-10-22	45000	7403	37597	336300	Random Forest	none	0.6507	0.0056	0.0028	1.0000	0.0017	0.9955
5	2020-11-27	45000	44960	40	328897	Random Forest	none	0.6942	0.0080	0.0032	0.6250	0.6650	0.3351
6	2021-03-16	45000	44375	625	283937	Random Forest	none	0.7522	0.0121	0.0025	0.5146	0.8266	0.1739
7	2021-08-06	45000	44609	391	239562	Random Forest	none	0.7652	0.0132	0.0026	0.4930	0.8559	0.1445
8	2022-05-13	46553	46501	52	194953	Random Forest	none	0.7707	0.0134	0.0023	0.5167	0.8472	0.1527
9	2022-07-28	82951	82919	32	148452	Random Forest	none	0.7503	0.0117	0.0023	0.5750	0.8089	0.1915
10	2023-05-26	128412	128379	33	65501	Random Forest	none	0.8349	0.0159	0.0021	0.6835	0.8218	0.1785
11	2024-06-21	91456	91424	32	19655	Random Forest	none	0.7863	0.0283	0.0022	0.4320	0.9343	0.0669
12	2024-09-30	19687	—	—	0	Random Forest	none	0.7372	0.0392	0.0032	0.3594	0.9446	0.0573
1	2019-06-11	45000	32	44968	391503	Random Forest	smote	0.7724	0.0119	0.0025	0.7544	0.6830	0.3168
2	2019-06-11	45000	383	44617	391471	Random Forest	smote	—	0.0000	0.0005	—	0.3750	0.6250
3	2019-06-12	45000	17582	27418	391088	Random Forest	smote	—	0.0000	0.0001	—	0.6449	0.3551
4	2020-02-28	45000	44609	391	373506	Random Forest	smote	0.7823	0.0083	0.0041	0.8500	0.5360	0.4633
5	2020-11-27	45000	44960	40	328897	Random Forest	smote	0.7187	0.0115	0.0045	0.7025	0.6724	0.3275
6	2021-03-16	45000	44609	391	283937	Random Forest	smote	0.7513	0.0120	0.0040	0.6154	0.7264	0.2733
7	2021-03-16	45000	32	44968	283546	Random Forest	smote	—	0.0000	0.0000	—	0.9062	0.0938
8	2021-03-16	45000	17933	27067	283514	Random Forest	smote	—	0.0000	0.0013	—	0.5938	0.4062
9	2021-08-31	168332	168288	44	149493	Random Forest	smote	0.7400	0.0119	0.0036	0.6104	0.7362	0.2636
10	2022-06-14	74676	74609	67	74348	Random Forest	smote	0.7938	0.0148	0.0022	0.5595	0.8484	0.1527
11	2022-08-25	123729	123683	46	69739	Random Forest	smote	0.7722	0.0137	0.0022	0.4722	0.8702	0.1303
12	2022-12-08	217534	217495	39	20327	Random Forest	smote	0.8075	0.0194	0.0021	0.5556	0.9051	0.0958
13	2022-12-24	214793	214754	39	20288	Random Forest	smote	—	0.0000	0.0000	—	0.8750	0.1250
14	2022-12-24	214793	214754	39	20249	Random Forest	smote	—	0.0000	0.0000	—	0.9375	0.0625
15	2022-12-24	214793	214754	39	20210	Random Forest	smote	—	0.0000	0.0011	—	0.8125	0.1875
16	2022-12-24	214793	214754	39	20171	Random Forest	smote	—	0.0000	0.0000	—	0.9062	0.0938
17	2023-05-12	200665	200624	41	20271	Random Forest	smote	—	0.0000	0.0027	—	0.6250	0.3750
18	2023-05-19	132790	132749	41	20230	Random Forest	smote	—	0.0000	0.0000	—	0.8750	0.1250
19	2023-05-19	132790	132749	41	20189	Random Forest	smote	—	0.0000	0.0000	—	0.9062	0.0938
20	2024-06-28	45000	383	44617	17361	Random Forest	smote	0.5864	0.0000	0.0026	0.0000	0.7094	0.2924
21	2024-07-01	45000	14072	30928	16978	Random Forest	smote	0.5916	0.0000	0.0026	0.0000	0.8194	0.1828
22	2024-09-20	45000	39695	5305	2906	Random Forest	smote	0.7561	0.0128	0.0030	0.6667	0.6944	0.3057
23	2024-09-30	8211	—	—	0	Random Forest	smote	0.6705	0.0119	0.0041	0.5833	0.5999	0.4002

Table A.11: Evaluation metrics for adaptive KSWIN drift detection

Model	Bal.	Drift Date	Pred. Year	Seg. Rows	Retrain Rows	AUC	PR-AUC	F1	Sens.	Spec.	Err. Rate
AdaBoost	none	2020-01-14	2020	52,384	2,197	0.6595	0.0061	0.0167	0.3015	0.9092	0.0924
AdaBoost	none	2020-08-14	2020	28,672	4,687	0.6202	0.0051	0.0131	0.1154	0.9550	0.0473
AdaBoost	none	2021-01-19	2021	41,191	8,380	0.7448	0.0105	0.0089	0.7653	0.5956	0.1381
AdaBoost	none	2024-01-01	2024	254,102	8,635	0.5550	0.0033	0.0051	0.8240	0.2128	0.3182
AdaBoost	none	2024-09-30	2024	60,065	0	0.6638	0.0047	0.0086	0.6039	0.6429	0.3572
AdaBoost	smote	2020-01-14	2020	52,384	2,197	0.6522	0.0052	0.0073	0.6176	0.5638	0.4360
AdaBoost	smote	2020-08-14	2020	28,672	4,687	0.5952	0.0050	0.0058	0.9744	0.0856	0.9120
AdaBoost	smote	2021-01-19	2021	41,191	8,380	0.6915	0.0074	0.0076	0.6122	0.6218	0.4006
AdaBoost	smote	2024-01-01	2024	254,102	8,635	0.6638	0.0040	0.0093	0.5625	0.7121	0.4868
AdaBoost	smote	2024-09-30	2024	60,065	0	0.6682	0.0110	0.0069	0.7987	0.4135	0.5855
NNet	none	2020-01-14	2020	52,384	2,197	0.4596	0.0025	0.0059	0.3088	0.7305	0.2706
NNet	none	2020-08-14	2020	28,672	4,687	0.5145	0.0035	0.0057	0.5513	0.4762	0.5236
NNet	none	2021-01-19	2021	41,191	8,380	0.3298	0.0017	0.0035	0.6327	0.1441	0.4052
NNet	none	2024-01-01	2024	254,102	8,635	0.4290	0.0020	0.0041	0.4424	0.4860	0.2617
NNet	none	2024-09-30	2024	60,065	0	0.5600	0.0036	0.0076	0.2792	0.8140	0.1874
NNet	smote	2020-01-14	2020	52,384	2,197	0.5857	0.0045	0.0098	0.1838	0.9051	0.0968
NNet	smote	2020-08-14	2020	28,672	4,687	0.5169	0.0035	0.0063	0.5128	0.5600	0.4402
NNet	smote	2021-01-19	2021	41,191	8,380	0.5280	0.0041	0.0048	0.5612	0.4460	0.3899
NNet	smote	2024-01-01	2024	254,102	8,635	0.5381	0.0026	0.0055	0.5033	0.5626	0.3681
NNet	smote	2024-09-30	2024	60,065	0	0.6565	0.0060	0.0058	0.9091	0.1979	0.8002
Random Forest	none	2020-01-14	2020	52,384	2,197	0.6634	0.0064	0.0130	0.3603	0.8588	0.1425
Random Forest	none	2020-08-14	2020	28,672	4,687	0.6371	0.0065	0.0072	0.5897	0.5549	0.0696
Random Forest	none	2021-01-19	2021	41,191	8,380	0.6994	0.0076	0.0084	0.5612	0.6844	0.2176
Random Forest	none	2024-01-01	2024	254,102	8,635	0.6890	0.0064	0.0088	0.6168	0.6686	0.3511
Random Forest	none	2024-09-30	2024	60,065	0	0.7232	0.0129	0.0083	0.7792	0.5232	0.4762
Random Forest	smote	2020-01-14	2020	52,384	2,197	0.6926	0.0081	0.0107	0.5515	0.7357	0.2648
Random Forest	smote	2020-08-14	2020	28,672	4,687	0.6693	0.0062	0.0099	0.6026	0.6713	0.8641
Random Forest	smote	2021-01-19	2021	41,191	8,380	0.6893	0.0072	0.0102	0.3980	0.8174	0.3027
Random Forest	smote	2024-01-01	2024	254,102	8,635	0.7181	0.0082	0.0084	0.8109	0.5417	0.4560
Random Forest	smote	2024-09-30	2024	60,065	0	0.7383	0.0121	0.0101	0.7208	0.6362	0.3636
XGBoost	none	2020-01-14	2020	52,384	2,197	0.6092	0.0051	0.0125	0.3162	0.8713	0.1302
XGBoost	none	2020-08-14	2020	28,672	4,687	0.6211	0.0053	0.0102	0.1410	0.9275	0.1018
XGBoost	none	2021-01-19	2021	41,191	8,380	0.7334	0.0246	0.0088	0.7959	0.5750	0.1689
XGBoost	none	2024-01-01	2024	254,102	8,635	0.6852	0.0047	0.0085	0.3602	0.8004	0.3707
XGBoost	none	2024-09-30	2024	60,065	0	0.7158	0.0181	0.0172	0.4351	0.8740	0.1271
XGBoost	smote	2020-01-14	2020	52,384	2,197	0.6276	0.0060	0.0094	0.4412	0.7600	0.2408
XGBoost	smote	2020-08-14	2020	28,672	4,687	0.6103	0.0069	0.0108	0.4231	0.7904	0.8832
XGBoost	smote	2021-01-19	2021	41,191	8,380	0.6808	0.0071	0.0072	0.5612	0.6340	0.3084
XGBoost	smote	2024-01-01	2024	254,102	8,635	0.5896	0.0035	0.0062	0.3832	0.7071	0.4576
XGBoost	smote	2024-09-30	2024	60,065	0	—	—	—	—	—	—

Table A.12: Global summary of average performance metrics across all prediction periods, by strategy, model, and class balancing mode.

Strategy	Model	Balance	AUC	PR-AUC	F1	Brier	Sensitivity	Specificity	Error Rate	N Segments
Static	AdaBoost	none	0.7610	0.0112	0.0155	0.0025	0.5651	0.8221	0.1786	6
Static	AdaBoost	smote	0.7492	0.0088	0.0117	0.0025	0.7193	0.6995	0.3004	6
Static	NNet	none	0.5065	0.0032	0.0062	0.0025	0.0996	0.9184	0.0836	6
Static	NNet	smote	0.7526	0.0176	0.0140	0.0024	0.6163	0.7861	0.2143	6
Static	Random Forest	none	0.7832	0.0160	0.0126	0.0025	0.7277	0.7187	0.2813	6
Static	Random Forest	smote	0.7875	0.0203	0.0116	0.0025	0.7605	0.6795	0.3203	6
Static	XGBoost	none	0.7785	0.0216	0.0097	0.0025	0.7873	0.5998	0.3997	6
Static	XGBoost	smote	0.7772	0.0213	0.0102	0.0025	0.7647	0.6327	0.3670	6
Period-Aligned	AdaBoost	none	0.7495	0.0149	0.0137	0.0025	0.6121	0.7164	0.2839	6
Period-Aligned	AdaBoost	smote	0.7483	0.0128	0.0146	0.0025	0.5899	0.7922	0.2083	6
Period-Aligned	NNet	none	0.4609	0.0024	0.0037	0.0025	0.3439	0.6244	0.3763	6
Period-Aligned	NNet	smote	0.7308	0.0166	0.0132	0.0025	0.6142	0.7590	0.2413	6
Period-Aligned	Random Forest	none	0.7804	0.0188	0.0134	0.0025	0.7237	0.6618	0.3380	6
Period-Aligned	Random Forest	smote	0.7794	0.0201	0.0127	0.0025	0.7148	0.7091	0.2909	6
Period-Aligned	XGBoost	none	0.7808	0.0202	0.0135	0.0025	0.7048	0.7335	0.2666	6
Period-Aligned	XGBoost	smote	0.7754	0.0182	0.0135	0.0025	0.6943	0.7304	0.2697	6
Cumulative	AdaBoost	none	0.7779	0.0186	0.0213	0.0025	0.5219	0.8711	0.1297	6
Cumulative	AdaBoost	smote	0.7660	0.0170	0.0165	0.0025	0.6079	0.8073	0.1932	6
Cumulative	NNet	none	0.5169	0.0033	0.0055	0.0025	0.4080	0.6374	0.3632	6
Cumulative	NNet	smote	0.7731	0.0207	0.0135	0.0024	0.6913	0.7443	0.2559	6
Cumulative	Random Forest	none	0.7955	0.0221	0.0166	0.0025	0.6527	0.8043	0.1961	6
Cumulative	Random Forest	smote	0.7967	0.0272	0.0149	0.0025	0.7096	0.7629	0.2372	6
Cumulative	XGBoost	none	0.7983	0.0269	0.0167	0.0025	0.6698	0.7852	0.2151	6
Cumulative	XGBoost	smote	0.7949	0.0266	0.0144	0.0024	0.7175	0.7452	0.2549	6
Adaptive KSWIN	AdaBoost	none	0.6487	0.0059	0.0007	0.0025	0.5220	0.8189	0.1811	6
Adaptive KSWIN	AdaBoost	smote	0.6542	0.0065	0.0005	0.0024	0.7131	0.5613	0.4386	6
Adaptive KSWIN	NNet	none	0.4586	0.0027	0.0004	0.0025	0.4429	0.8531	0.1469	6
Adaptive KSWIN	NNet	smote	0.5650	0.0042	0.0004	0.0024	0.5340	0.7697	0.2303	6
Adaptive KSWIN	Random Forest	none	0.6824	0.0080	0.0006	0.0024	0.5815	0.9100	0.0900	6
Adaptive KSWIN	Random Forest	smote	0.7015	0.0084	0.0007	0.0025	0.6167	0.7365	0.2636	6
Adaptive KSWIN	XGBoost	none	0.6729	0.0116	0.0008	0.0025	0.4097	0.8520	0.1481	6
Adaptive KSWIN	XGBoost	smote	0.6481	0.0082	0.0006	0.0025	0.5072	0.7791	0.2209	6
Adaptive ADWIN	AdaBoost	none	0.7010	0.0167	0.0122	0.0025	0.6401	0.7119	0.2875	25
Adaptive ADWIN	AdaBoost	smote	0.6757	0.0088	0.0087	0.0032	0.5081	0.7090	0.2912	14
Adaptive ADWIN	NNet	none	0.4733	0.0035	0.0042	0.0021	0.2297	0.7402	0.2610	19
Adaptive ADWIN	NNet	smote	0.6680	0.0147	0.0154	0.0022	0.4628	0.7974	0.2033	17
Adaptive ADWIN	Random Forest	none	0.7519	0.0158	0.0162	0.0037	0.6087	0.7279	0.2724	12
Adaptive ADWIN	Random Forest	smote	0.7451	0.0516	0.0114	0.0050	0.6703	0.6723	0.3270	26
Adaptive ADWIN	XGBoost	none	0.7232	0.0418	0.0101	0.0017	0.7067	0.6580	0.3419	19
Adaptive ADWIN	XGBoost	smote	0.6465	0.0141	0.0053	0.0036	0.5843	0.7425	0.2576	28
Adaptive ARF	ARFfast	none	0.4688	0.0022	0.0008	0.0025	0.0569	0.9375	0.0647	6
Adaptive ARF	ARFmaj	none	0.4711	0.0022	0.0012	0.0025	0.0578	0.9341	0.0681	6
Adaptive ARF	ARFmoderate	none	0.4711	0.0022	0.0012	0.0025	0.0578	0.9341	0.0681	6

Acknowledgments

Franco Basso gratefully acknowledges the financial support from the Complex Engineering Systems Institute, ISCI (ANID-PIA/PUENTE AFB230002), ANID-FONDECYT 1241335, ANID-FONDEF ID25I10409 and ANID-FOVI 240048. Raúl Pezoa acknowledges the financial support from ANID-FONDECYT 1241913. Mauricio Varas acknowledges the financial support from ANID-FONDECYT 1241843. Franco Basso, Diego A. Martinez and Vicente Fernández-Figueroa gratefully acknowledge the financial support from PUCV project 039.309/2024.